# React Clean Architecture

Daichi Imamori

## Redefining React application design in the context of Clean Architecture!

1. What is Clean Architecture?
2. What is scalable design?
3. What is a change resistant design?

# React Clean Architecture

## Redefining React application design in the context of Clean Architecture

# React Clean Architecture

## Redefining React application design in the context of Clean Architecture

Daichi Imamori

# Introduction

Thank you for taking the time to read this book.You're reading this book, that means you're probably interested in Clean Architecture, React application development, or both.Clean Architecture has been gaining attention as a methodology for product design since it was published by Robert C. Martin in August 2012.Since this approach targets the entire product development, it has traditionally been used mainly for backend development and native app development.However, with the recent development of more sophisticated browsers and JavaScript-related technologies, technologies such as SPA and WPA have become popular. This has led to an increase in the number of advanced applications being built using the web frontend alone.Also, with the ability to build fully managed and serverless backends like Firebase and AWS Amplify, the responsibility is increasingly on the web frontend.React and Redux are two of the major technologies that support this continuous development of web frontend technology.In this book, we will review the design of applications built with React and Redux from the perspective of Clean Architecture, and sometimes propose new designs.The goal is to apply this methodology to the Web frontend domain, where Clean Architecture has not been widely adopted until now.Of course, Clean Architecture is not a silver bullet that solves everything, and how it should be applied depends on the target product and team.I hope this book will help you get one step ahead in React application design.

# A step ahead in React application design

This document begins with an explanation of the background of Clean Architecture and its basic principles.In particular, each principle of SOLID and dependency constraints will be explained in detail.One chapter is devoted to explaining Dependency Injection (DI) using a concrete implementation.After reviewing the basics of Clean Architecture, we will reconsider the general design of SPA and React from the perspective of Clean Architecture.In this context, we propose "Redux Concentric Diagram" and "Component Concentric Diagram".

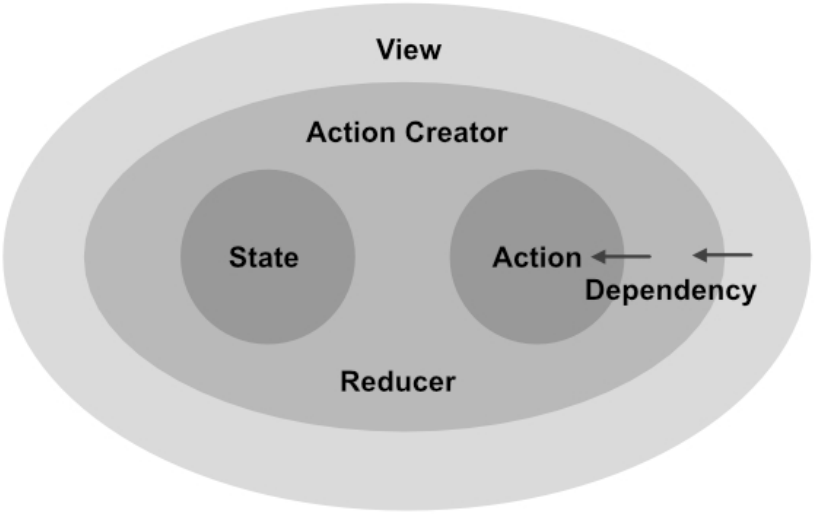


**Figure 1: Redux Concentric Circles**

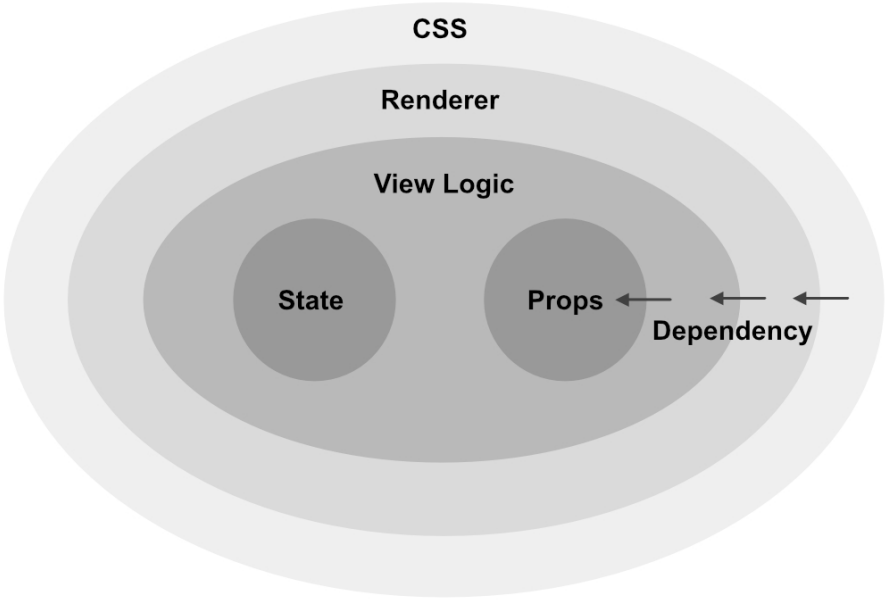


**Figure 2: Component Concentric Circles**

In this way, by looking at React and Redux from the perspective of Clean Architecture,I expect for you to learn about aspects that you hadn't noticed before.

---

# Intended audience

This book was written for an audience of people who have experience developing React applications.For this reason, I will not explain the technical details of TypeScript, React, and Redux, but will assume that you already have some knowledge of them.Clean Architecture is explained in terms of background and basic principles, so even those who are new to the subject can read on.

# Notation

In this book, a book name is written with double quotation "".In addition, a book nam will be abbreviated by one sentence in the text.For example, "Clean Architecture: A Craftsman's Guide to Software Structure and Design" should be written as "Clean Architecture".

# Table of Contents

# Chapter 1. Clean Architecture

Throughout this book, we will discuss how to apply Clean Architecture to React applications. In the process, some of the principles of Clean Architecture will be followed, while others will be ignored. This is because, as we will see later, Clean Architecture is an abstraction of best practices extracted from empirical rules, and it is not necessarily good to follow all of them faithfully. In other words, it is important to select the rules that should be followed depending on the product and team to which it is applied. In this book, I will explain the main rules and suggests how to apply them to React application development. In this chapter, I will explain the principles of Clean Architecture. If you are already familiar with Clean Architecture, you can skip this chapter.

## 1.1. What is Clean Architecture?

Clean Architecture is a software design methodology proposed by Robert C. Martin.Looking back at its history, the Clean Architecture methodology was introduced by a blog post on August 13, 2012*1 .An English version of the book was published in 2017.

[*1] https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html

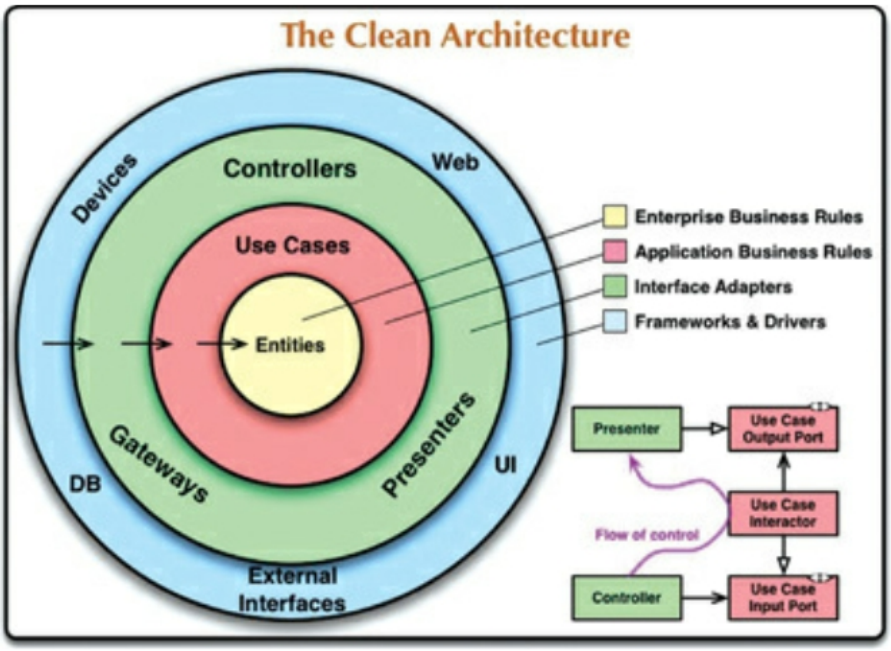


**Figure 1.1: Clean Architecture concentric circles*2**

[*2] Robert C, Martin. "The Clean Architecture". Clean Architecture. Pearson Education.

Clean Architecture is a design methodology derived from Robert C. Martin's experience in various software (or hardware) developments.The image Figure 1.1 is often used to represent Clean Architecture, and even if you don't know much about Clean Architecture, you've probably seen this diagram before.The details of this diagram will be described later, but I introduced it at the beginning because it is an important diagram in explaining the Clean Architecture.I hope you will keep this concentric circle diagram in mind as you continue reading.

The following sentence is from the preface of Clean Architecture.

> **The architecture rules are the same!**
>
> This is startling because the systems that I have built have all been so radically different.Why should such different systems all share similar rules of architecture?My

> conclusion is that the rules of software architecture are independent of every other variable.
>
> Robert C, Martin. "Preface". Clean Architecture. Pearson Education.

These common rules for software development are summarized as practices and are called Clean Architecture.In other words, Clean Architecture was born as a common rule by highly abstracting the experience gained from various developments.

Of course, extracting best practices from experience and compiling them into rules has been done before Clean Architecture.The history of programming language paradigms is just that, and so are programming patterns such as the GoF design pattern.There is also a collection of design principles called SOLID, compiled by Uncle Bob himself *3 .

[*3] Not all of SOLID's principles were suggested by Uncle Bob.

In "Clean Architecture", Sections I through IV will be used to explain these traditional practices.In this way, Clean Architecture is a design methodology that draws on existing practices.Therefore, Clean Architecture is not a design methodology that has been released in recent years, but rather it is based on empirical rules that have been developed by many software developers over the years.

On the other hand, there are some things to keep in mind.As mentioned earlier, Clean Architecture is a highly abstracted method.Since it does not propose specific implementation methods, it cannot be directly applied to actual software development.Therefore, it is necessary to understand the meaning of the abstraction of this design methodology and to put it into the concrete work of actual software development.In some cases, you may want to apply some of the rules in Clean Architecture and ignore others.However, even if you dare to ignore the rule, there is a big difference between knowing the rule and ignoring it, and breaking the rule without knowing it.In this book, we will grasp the principles of

Clean Architecture and then discuss how to apply Clean Architecture when building React applications.

### Robert C. Martin

Robert C. Margin, also known as Uncle Bob, is an software engineer.He has been a software developer since the 1970s, with experience in a variety of software development, and has published many technical articles and books.Some of his notable publications include

- 2002. Agile Software Development, Principles, Patterns, and Practices. ISBN 978-0135974445.
- 2009. Clean Code: A Handbook of Agile Software Craftsmanship. ISBN 978-0132350884.
- 2011. The Clean Coder: A Code Of Conduct For Professional Programmers. ISBN 978-0137081073.
- 2017. Clean Architecture: A craftsman's Guide to Software Structure and Design. ISBN 978-0134494166.
- 2019. Clean Agile: Back to Basics. ISBN 978-0135781869.

He contribute technical articles to a blog called The Clean Code Blog *4 .He is also the founder and president of Object Mentor, a company whose main business is to provide development support to clients.Thus, he is a person who is well versed in the organizational and design theories of software development and works to propagate these theories through his publications and corporate activities.

[*4] https://blog.cleancoder.com/

# 1.2. Why Clean Architecture?

In "Clean Architecture", the world realized by good software design is described as follows

> The goal of software architecture is to minimize the human resources required to build and maintain the required system.
>
> Robert C, Martin. "What Is Design and Architecture?". Clean Architecture. Pearson Education.

Leaving aside the question of how feasible this goal is, few developers would deny the goal itself.Clean Architecture has been proposed as one of the means to achieve this goal.As I said it is a means to an end, you can also aim for this goal by means other than Clean Architecture.Of course, not everything will work with Clean Architecture.The specific implementation is not within the scope of Clean Architecture, and adopting it does not immediately solve anything.What Clean Architecture provides is the thinking and practices to achieve the above goals.There is no solid right answer for the implementation to realize it, and there are as many right answers as there are products.

Learn about the background and concept of Clean Architecture, and we believe that cultivating the ability to think about how to apply it to specific implementations is necessary to achieve a step-change in software design.In particular, it can be applied to React applications to empower not only web development but also application development on multiple platforms and it will support product growth.

## 1.3. Clean Architecture and Domain Driven Design

Domain Driven Design (DDD) is one of the building blocks of Clean Architecture.DDD is a software design methodology proposed by Eric Evans.DDD focuses on how to link business domains and software development, and does not refer to individual technologies.What is important in DDD is that domain experts and software developers, who are familiar with the business domain, can communicate with each other through a common language, the ubiquitous language.The point is that we shape our services through communication.Clean Architecture is based on the concept of DDD. Domain models and domain logic, which are formed through communication, are the core and most important elements of a service.Clean Architecture focuses on how to protect them from surrounding dependencies.

In this way, Clean Architecture is closely related to DDD and cannot be separated from it.In the book "Clean Architecture", terms such as domain, domain model, and other terms used in DDD such as Entity and Repository also appear.

## 1.4. Design principles

In "Clean Architecture", before moving on to the discussion of architecture, there is an explanation of a design principle called SOLID.SOLID principles are a set of principles that apply at the module level and provide guidance for connections between modules.Although the design guidelines for the code level, which is the building block of a module, are not covered in this book, "Clean Code*5 " by Uncle Bob or "The Art of Readable Code*6 " may be

helpful.Adherence to the SOLID principles is a key element in achieving Clean Architecture.In this book, we will also look back at the content of SOLID before discussing Clean Architecture in React applications.

[*5] Clean Code: A Handbook of Agile Software Craftsmanship, ISBN 978-0132350884

[*6] The Art of Readable Code: Simple and Practical Techniques for Writing Better Code, ISBN 978-0596802295

SOLID is an acronym for the following five principles*7

- Single Responsibility Principle (SRP)
- Open-Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)

In Clean Architecture, the SOLID principle is introduced as having the following three objectives

- Be resistant to change.
- Easy to understand.
- Be available for use in many software systems as a basis for components

"Easy to understand" is a characteristic that is also required at the code level.Then, by connecting modules that consist of well-written and clean code based on SOLID, the relationship between modules becomes easy to understand.

At first glance, "Be available for use in many software systems" tends to be used in situations where many users will use it, such as

libraries and frameworks, but it can also be important in designing specific software.As your business succeeds and your software grows, you will increasingly want to make certain modules available to other systems within your business.For example, a service that was initially intended only for the web, but later grew to require native app development.If you want to use some modules of your software written in React from React Native, you can minimize the modification for reuse if the SOLID is satisfied beforehand.

And "Be resistant to change" is a key feature in carrying out the objective of protecting the business domain from changes in other components.In particular, the Dependency Inversion Principle (DIP) plays an important role in protecting the business domain from dependencies.Let me introduce the five principles of SOLID in turn.

[*7] SOLID was first published in 2000 in the paper *Design Principles and Design Patterns* .At that time, the name SOLID was not used, but in 2004, at the suggestion of Michael Feathers, the name SOLID came into use

## Single Responsibility Principle (SRP)

> A module should have one, and only one, reason to change.
>
> Robert C, Martin. "SRP: The Single Responsibility Principle". Clean Architecture. Pearson Education.

The term "actors" refers to a grouping of users and stakeholders.For example, let's take an e-commerce service as an example.The following actors may be considered.

- Users who shopping

- Users on the store side who manage the products
- Stakeholders who determine the commission to the store side
- The team that decides on the sale launch

For these actors, make sure that one module depends on one actor.In other words, SRP is the principle that no module should be affected by more than one actor.The DRY (Don't repeat yourself) principle should also be considered when applying SRP.I won't go into the details of the DRY principle itself, but in a nutshell, it is the principle that code that does the same thing should be put in one place.If you try to practice DRY easily, you will end up sharing the same process of doing the same thing.

If you make processes that have responsibilities for different actors common because they are doing similar things, you may have trouble later on.This is because even if they seem to be doing the same thing at the moment, as long as the actors they depend on are different, it may be necessary to add different actor-dependent processes in subsequent updates.In other words, you may end up merging seemingly identical but essentially different processes.You can distinguish such cases by identifying which actor the process depends on.Of course, even if the actors are different, the process can be common if there is a relationship between them.In any case, when practicing DRY, it is necessary to consider the SRP as well and make a careful decision.

## Open-Closed Principle (OCP)

The Open-Closed Principle (OCP) was proposed by Bertrand Meyer in 1988.

> A software artifact should be open for extension but closed for modification.
>
> Bertrand Meyer. Object Oriented Software Construction, Prentice Hall, 1988, p. 23.

It must be open for extensions such as adding features."open" means that it can be added without affecting the existing code.And when modifying the existing code of a module, the effect of the modification must be closed within the module.The OCP defines these two rules.

What are the situations in which this principle is not being followed?Even if you want to add a trivial feature, you will need to modify the existing code.When adding a new feature, we carefully read the dependencies among the thousands or tens of thousands of existing lines of code and make modifications.The larger the code base, the more man-hours it takes to add features, even if it is a trivial feature.On the other hand, if you follow OCP, you can limit the impact of specification changes and bug fixes because the modifications are closed to the module.

Being open to extensions and closed to modifications makes the software resistant to feature additions and specification changes.

## Liskov Substitution Principle (LSP)

The Liskov Substitution Principle (LSP) was proposed by Barbara Liskov in 1988.

> What is wanted here is something like the following substitution property:If for each object o1 of type S there is an object o2 of type T such that for all programs P defined

> in terms of T,the behavior of P is unchanged when o1 is substituted for o2 then S is a subtype of T.1
>
> Barbara Liskov, “Data Abstraction and Hierarchy,” SIGPLAN Notices 23, 5 (May 1988).

In other words, "If S is a derivative of T , then it must behave the same even if T is replaced by S .Since it is difficult to explain with just words, here is an example code.This can be expressed in TypeScript as follows.

List 1.1: Liskov Substitution Principle expressed in TypeScript

```
 1:    interface T {
 2:      value: number;
 3:    }
  4:    interface S extends T {}
 5:
 6:    const o1: S = { value: 1 };
 7:    const o2: T = { value: 2 };
 8:
 9:    const P = (arg: T) => arg.value;
10:    P(o2);
11:    P(o1);
```

Here, for the sake of illustration, we have assumed that `T` is a simple type with only `value` . In fact, what details `T` has has nothing to do with LSP.S inherits from `T` .The function `P` takes `T` as an argument.In this case, since `o2` is type `T` , so `P(o2)` is accepted.Similarly, `o1` , which is type `S` , `P(o1)` is also accepted.In such a case, we say that S is a derived type of `T` .In other words, the LSP says that if you want to inherit a type, it should be a derived type.

Looking at this example alone, one might think, "What is this, just inheritance of types, or is LSP just a rephrasing of the language feature of inheritance?To show that this is wrong, here is an example of inheritance that violates the LSP.

List 1.2: example of violating LSP

```
 1:    class Rectangle {
 2:      set width(w: number) {
 3:        this.width = w;
 4:      }
 5:
 6:      set height(h: number) {
 7:        this.height = h;
 8:      }
 9:
10:      calcArea(): number {
11:        return this.width * this.height;
12:      }
13:    }
14:
15:    class Square extends Rectangle {
16:      set width(w: number) {
17:        this.width = w;
18:        this.height = w;
19:      }
20:
21:      set height(h: number) {
22:        this.width = h;
23:        this.height = h;
24:      }
25:    }
26:
27:    const o1 = new Square();
28:    const o2 = new Rectangle();
29:
30:    // Function to find the ratio of the
sum of the lengths of the sides to the area
```

```
31:    const P = (arg: Rectangle, width,
height) => {
32:      arg.width = width;
33:      arg.height = height;
34:      return (width * 2 + height * 2) /
arg.calcArea();
35:    };
36:
37:    P(o2, 2, 5); // Can be calculated
correctly
38:    P(o1, 2, 5); // 5 * 4 / 5 * 5 is
correct, but actually (2 * 2 + 5 * 2) / 5 * 5
is calculated
```

In List 1.2 , we use the `Rectangle` type to represent a rectangle and the `Square` type to represent a square as examples.This is an example of an old known LSP violation.Mathematically, according to the definition, a square is a special case of a rectangle, so it might seem natural that `Square` inherits from `Rectangle` .However, a square assumes tighter constraints than a rectangle. It is the constraint that the length of the vertical side is equal to the length of the horizontal side.Therefore, in `Square` , when setting the length of the edge, the length of the vertical and horizontal edges are processed to be the same.As a result, `P(o1, 2, 5)` is getting a different result than intended.

If there is codes that violates the LSP like this, conditional branching will occur on the client side that uses it (in this case, the function `P( )` ).Then, when there are more classes inheriting from `T` , it becomes necessary to modify the conditional branch on the client side.This means that it also violates the Open-Closed Principle (OCP).

## Interface Segregation Principle (ISP)

The Interface Segregation Principle (ISP) can be summed up: You should not cram everything into a single interface, but provide only the functionality that the client needs.

This is also related to SRP.Consider the following example of an EC site with the following actors.

- `Buyer` : The user who purchases the product
- `Seller` : The user who sells the product

Then, these two users are abstracted by a single interface `User` .`User` requests the methods `getPurchaseHistory()` and `getTotalSales()` , but the former method is only used by `Buyer` , while the latter is only required by `Seller` .At this time, the implementation of `Buyer` will need to implement some processing, even though it does not use `getTotalSales()` .To begin with, `Buyer` did not need to know about the existence of `getTotalSales()` , but by violating ISP, it created an unnecessary dependency.`User` also violates SRP because it depends on two actors.Using or modifying `User` will require changes to both `Buyer` and `Seller` , which may also violate OCP.

Being careful not to overburden any one interface according to the ISP will also help to protect other principles such as SRP and OCP.

List 1.3: example of multiple actors relying on a common interface

```
 1:    interface User {
 2:      id: string;
 3:      name: string;
 4:      getPurchaseHistory: () => Purchase[];
// get the purchase history
 5:      getTotalSales: () => number; // get
total sales
 6:    }
```

```
 7:
 8:    class Buyer implements User {
 9:      constructor(public id: string, public
name: string) { }
10:
11:      getPurchaseHistory() {
12:        const history = ...; // Process to
retrieve purchase history
13:        return history;
14:      }
15:
16:      getTotalSales() {
17:        return 0;
18:      }
19:    }
20:
21:    class Seller implements User {
22:      constructor(public id: string, public
name: string) { }
23:
24:      getPurchaseHistory() {
25:        return [];
26:      }
27:
28:      getTotalSales() {
29:        const sales = ...; // Process to
calculate total sales
30:        return sales;
31:      }
32:    }
```

## Dependency Inversion Principle (DIP)

The last part of SOLD is Dependency Inversion Principle (DIP).

> The Dependency Inversion Principle (DIP) tells us that the most flexible systems are those in which source code dependencies refer only to abstractions, not to concretions.
>
> Robert C, Martin. "DIP: The Dependency Inversion Principle". Clean Architecture. Pearson Education.

What is the source code dependencies?In the case of TypeScript, a dependency is created by referencing `import` or `require` .In other words, the DIP says that when referring to `import` , you should specify an abstract code such as `interface` , not a concrete implementation code.

For example, suppose that `a.ts` imports `b.ts` .At this time, `a.ts` will know the file name of `b.ts` as well as the classes and methods described in it.And if `b.ts` is changed, it will also affect `a.ts` .This is even if `a.ts` is originally a stable code that is not easily changed.

## stability of the code

This is where the idea of "stable code" comes into play.Suppose all the codes could be divided into "stable codes" and "unstable codes".Then, the dependencies can be classified into four types as follows

1. stable → stable
2. stable → unstable
3. unstable → stable
4. unstable → unstable

Think of the left side of the arrow as being dependent on the right side.The first is not a problem because both sides are stable.In 2,

the stable code depends on the unstable code, and the inherent stability is compromised.3 does not cause problems because the unstable code depends on the stable code.In 4, the unstable code depends on the unstable code, which makes it more unstable, but it is not as much of a problem as 2.In other words, we need to eliminate the relationship in 2 and reduce the relationship in 4 as much as possible.

So what exactly are "stable code" and "unstable code"?In Clean Architecture, we focus on the following two points when considering stability.

1. abstract or concrete
2. close to input/output or close to domain model

**Abstract or concrete?**

Hidden in the DIP is the assumption that abstract components are stable and concrete components are unstable.In order to verify whether this assumption is correct, let's compare the abstract interface with its concrete implementation.As the interface is modified, the implementation is modified accordingly.On the other hand, the interface does not necessarily need to be changed when the implementation is changed.This shows that the abstracted code is more stable.It can be said that abstraction is the process of extracting the common denominator or essence from multiple concrete objects. Therefore, it is easy to understand that abstracted code is hard to be changed.However, it is important to note that implementation is not always easy to change.For example, the standard library of a programming language is an implementation, but it does not change much.There is no need to be more nervous than necessary about references to such an unchanging implementation.

Thus, abstract or concrete is an important indicator for the stability of a code.On the other hand, concrete codes are not necessarily

changeable.In addition, please keep in mind that not all codes can be divided into stable or unstable, and that there are varying degrees of stability there.

### Close to input/output or close to domain model

As briefly introduced in Figure 1.1 , Clean Architecture divides the software into layers of concentric circles.The UI View, DB, storage and other inputs and outputs are located on the outermost side, and the domain model is located on the innermost side.The intention is to place what we want to stabilize inside the layer and protect it from the unstable outside.We also use DIP to protect the center of the circle, which we want to stabilize.This concentric circle architecture is the fundamental idea of Clean Architecture.This is supported by DIP.

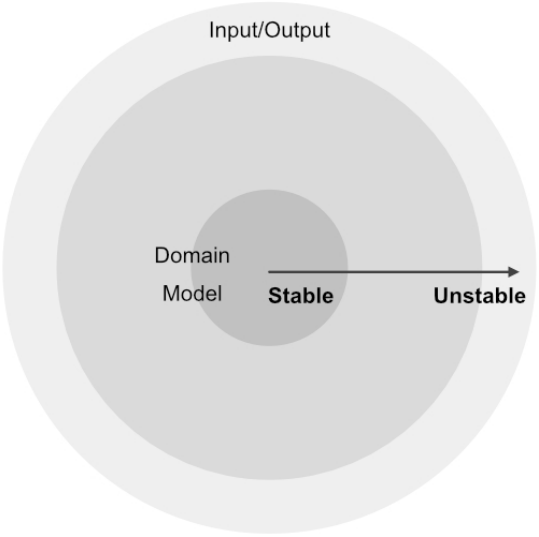


**Figure 1.2: Concentric circles and the relationship between stability and instability**

## Dependency Inversion

According to the DIP, the destination referred to by `import` must be stable.But can we always follow this rule?There are many cases where you want to call unstable code from stable code for processing.For example, the Clean Architecture considers DB to be

the outermost of the concentric circles and unstable.If you want to operate DB from a stable code inside the circle, you need to operate an unstable code from a stable code.As you can see, the flow of processing and the direction of dependencies are usually the same. In other words, if you try to manipulate the DB, you will depend on the DB.However, this would violate the DIP. We need to reverse the process flow and dependencies somehow.Dependency Injection (DI) solves this problem.For more information about DI, please refer to Chapter 2"Dependency Injection" for a detailed explanation with sample codes.DI reverses the flow of processing and dependencies, and protects the domain model at the center of the concentric circles from instability.

---

# 1.5. Component Principles

This section describes the component-level design principles.As mentioned earlier, SOLID is a design principle for connections between modules.In Clean Architecture, a component is a set of modules, which is the unit of deployment.In other words, a component is a collection of modules designed based on SOLID.The component principles described in this section guide how to design components.Some of the principles of the component are similar to SOLID.This suggests that the main ideas of these principles can be applied recursively regardless of the size of the component.

### What is a component?

> Components are the units of deployment.They are the smallest entities that can be deployed as part of a system.In Java, they are jar files. In Ruby, they are gem files. In .Net, they are DLLs.In compiled languages, they are aggregations of binary files.In interpreted languages, they are aggregations of source files.In all languages, they are the granule of deployment.
>
> Robert C, Martin. "Components". Clean Architecture. Pearson Education.

In TypeScript, for example, each package managed by npm is a deployment unit and a component.Web Components, as the name implies, are also components.What is difficult to determine is what is a component in an application development.If you divide the compilation granularity appropriately into jar files as in a Java application, they become components.On the other hand, TypeScript applications are usually transpiled as a whole.In other words, an application is a single deployment unit.Of course, we may break it up into chunks to optimize delivery, but that is more to reduce the load on the client side rather than to reduce the granularity of build and deployment.With this background, it will be difficult to adopt the "component = deployment unit" approach as is.On the other hand, this doesn't mean that the component principle is useless for designing React applications.For example, in developing an application, processes that are highly reusable may be separated as libraries.When this happens, it means that the library is recognized as a separate component from the application.Have you ever made a library without being sure, because "it's highly reusable" or "it looks better as a library"?By knowing these principles, you will be able to determine how components should be made independent from the application.

## Reuse/Release Equivalence Principle (REP)

> The granule of reuse is the granule of release.
>
> Robert C, Martin. "Component Cohesion". Clean Architecture. Pearson Education.

The Reuse/Release Equivalence Principle (REP), as the name implies, is the principle that granule of reuse and granule of release should be equivalent.As an example, consider the reuse of a component A. Suppose that the desired function cannot be reused without another component B.In this case, the unit of reuse and the unit of release are different.This is an example of "unit of reuse < unit of release".On the other hand, if "unit of reuse > unit of release", then a single release unit contains multiple granule of reuse.In this case, if you modify only one reuse unit, you will also need to release it.From the above, by making the unit of reuse and the unit of release equivalent, the cohesion of components can be improved.

## Common Closure Principle (CCP)

> Gather into components those classes that change for the same reasons and at the same times.Separate into different components those classes that change at different times and for different reasons.
>
> Robert C, Martin. "Component Cohesion". Clean Architecture. Pearson Education.

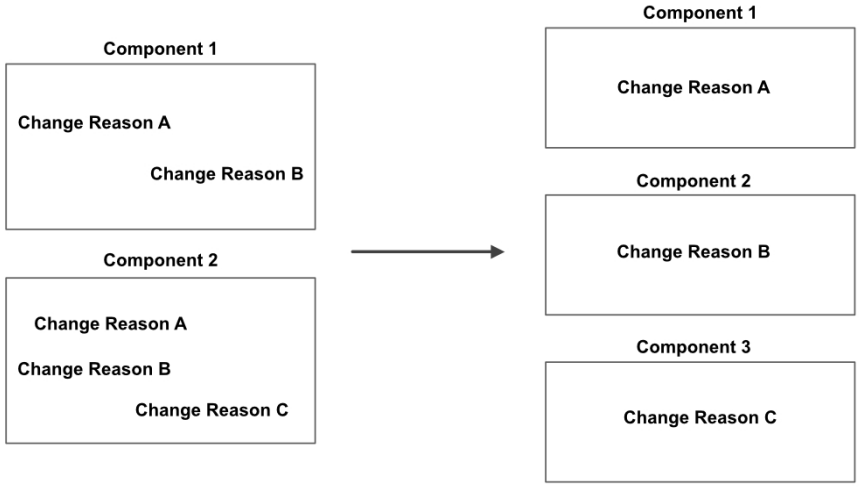


**Figure 1.3: Reconfigure components according to CCP with change reason**

The Common Core Principle of Closure (CCP) is a component version of the Single Responsibility Principle (SRP).What the two principles have in common is that one element should not be changed for more than one reason.In the case of SRP, an element is a class, and in the case of CCP, it is a component.

CCP is also closely related to the Open-Closed Principle (OCP).The "closed" in OCP meant closed to modification.CCP will change this wording to "combine similar types of changes into one component" to achieve the same goal.We aim to achieve the same goal.

When you want to cut out a library from your application, you need to pay attention to when that library is going to be modified.For example, if you want to cut out common UI components to be used in multiple applications, be careful that these UI components are really changed for different reasons than the application.You also need to pay attention to whether that each element of cut out UI component will be modified for the same reason.This is because elements that change for different reasons need to be separated into different components.

## Common Reuse Principle (CRP)

> Don’t force users of a component to depend on things they don’t need.
>
> Robert C, Martin. "Component Cohesion". Clean Architecture. Pearson Education.

The Common Reuse Principle (CRP) is a principle for determining which elements should be combined into components in the same way as CCP.While CPP focuses on the reason for the change, CRP focuses on the user's usage.It is also a generalization of Interface Separation Principle (ISP) to components.What CRP and ISP have in common is that users should not rely on what users do not use.

CRP can help you determine not only which elements should be included in a component, but also which elements should not be included.By assuming the user who will use the component, we can ensure that the component does not contain elements that will not be used by that user.

In general, I think there are multiple possible use cases for a component.For example, there are libraries for date manipulation such as Moment.js *8 .This library is divided into two components: Moment and Moment Timezone.This is because there are two use cases where you want to perform date manipulations: one where you want to take timezone into account, and one where you do not.If we follow the CRP approach, we should avoid forcing users who do not need timezone handling to rely on the timezone module.In other words, assuming that the use case of not using timezone is not negligible, this kind of component division in Moment.js satisfies the CRP.

[*8] Moment.js has been announced as going into maintenance mode in September 2020. https://momentjs.com/docs/#/-project-status/

## Acyclic Dependencies Principle (ADP)

> Allow no cycles in the component dependency graph.
>
> Robert C, Martin. "Component Coupling". Clean Architecture. Pearson Education.

Acyclic Dependency Principle (ADP), as the name implies, calls for dependencies to be acyclic.In TypeScript, it can be detected by eslint rule `import/no-cycle` .It is easy to imagine that the cycle of dependency can lead to many unexpected problems.For example, if class A and class B depend on each other, it is easy to detect and imagine the possible problems.In an extreme case, if class A and class B instantiate each other in the constructor, the process will loop.A little more confusing is the case of indirect circular dependence. In this case, the problem is that a modification of one side indirectly affects the other.Suppose that class C and class D are indirectly circularly dependent on each other. Then, changes to class C will have some effect on class D.Then class D will be modified to accommodate the changes in class C.In this case, class C also depends on class D, so class C is also affected by the modification of class D.This is even though the modifications to class D were intended to address changes in class C.Thus, the cyclic of dependencies means that the effects on change are cyclic.For products with larger applications and more developers, these issues can be hard to ignore.

In order to solve the circular dependency, we need to rethink the division of components.Specifically, we will cut out the elements that class C and class D have in common and make them into new components.

## Stable Dependencies Principle (SDP)

> Depend in the direction of stability.
>
> Robert C, Martin. "Component Coupling". Clean Architecture. Pearson Education.

Stable Dependency Principle (SDP) defines the relationship between dependency and stability.As explained in SOLID's Dependency Inversion Principle (DIP), code stability and dependency direction are closely related.SDP gives the details of the relation.

## Stability

So how is the stability level determined?In Clean Architecture, stability is defined as "something that cannot be easily moved".A component that is "hard to change" is said to be a stable component.In other words, it is dependent on many components.This is because if you change a component that is dependent on many other components, it will affect many other components and they will need to be adjusted.The number of dependencies from other components determines the stability of that component.If you want a component to be stable, design it to be dependent on other components.On the other hand, components that are expected to change a lot need to be easy to modify, and thus need to be unstable.To make a component unstable, make it as independent from other components as possible.

In "Clean Architecture", a quantitative measure of stability is presented.If you are interested in this topic, please refer to "Clean Architecture".

## Stable Abstractions Principle (SAP)

> A component should be as abstract as it is stable.
>
> Robert C, Martin. "Component Coupling". Clean Architecture. Pearson Education.

Stable Abstractions Principle (SAP) specifies the relationship between stability and abstraction.In the previous section, we explained that stability can be specified by dependencies.And SAP says that the level of abstraction must be determined by the level of stability.So why does a highly stable component also need to be highly abstract?As an example, let's look at the business domain.Your business domain is the core element of your business, and you want it to be stable.Therefore, it should be designed to be dependent on other components to ensure a high degree of stability.But then, the high stability of the business domain makes it difficult to fix.This makes the business inflexible, and the architecture becomes a bottleneck for the business.So, to make it easier to make changes while maintaining a high level of stability, we use a high level of abstraction.Design the business logic to a high level of abstraction through interfaces and abstract classes, and make other components depend on these.This way, modifying the business logic within that level of abstraction will not affect other components.

## SDP + SAP = DIP

Stable Dependency Principle (SDP) says, "Depend on the less stable to the more stable".Stable Abstractions Principle (SAP) says, "If something is less stable, make it less abstract. If it is more stable, make it more abstract".Putting these two principles together, we can say, "Depend on the lower level of abstraction to the higher level".This is exactly what Dependency Inversion Principle (DIP) is.Of course, DIP is a module-level principle, while SDP and SAP are component-level principles, so strictly speaking, they cannot be put together.However, through the explanations I have given so far, I

think the relationship between these three principles has become clearer.

Finally, Table 1.1 shows the correspondence between SOLID and the component principles.

**Table 1.1: correspondence between SOLID and component principles**

| Module level | Component level |
|---|---|
| Single Responsibility Principle (SRP) | Common Closure Principle (CCP) |
| Interface Segregation Principle (ISP) | Common Reuse Principle (CRP) |
| Dependency Inversion Principle (DIP) | Stable Dependencies Principle (SDP)<br>+ Stable Abstractions Principle (SAP) |
| Open-Closed Principle (OCP) | Common Closure Principle (CCP) |
| Liskov Substitution Principle (LSP) | - |

---

# 1.6. Clean Architecture

In this section, we will finally discuss the Clean Architecture.As we have discussed in this chapter, Clean Architecture focuses on how to protect the core business domain of the product from other dependencies.Decouple the business domain from specific and changeable technologies such as UI View, DB, and storage.By doing so, you can protect critical business rules from change and separate business decisions from technology factors.So how exactly do you protect your business domain from change?In this section, we will answer that question.

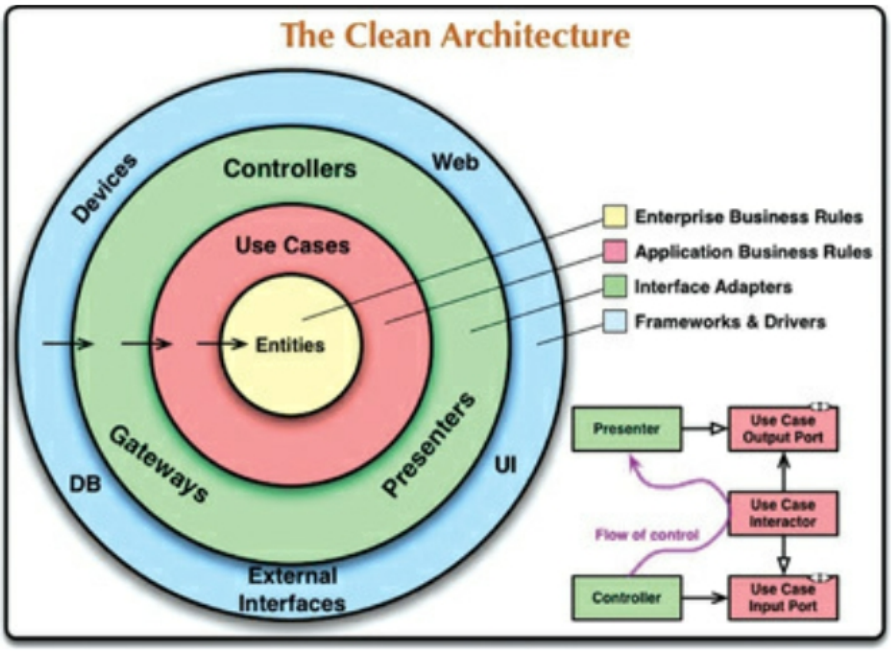


**Figure 1.4: (Re-post)Clean Architecture concentric circles*9**

[*9] Robert C, Martin. "The Clean Architecture". Clean Architecture. Pearson Education.

The concentric circles in Figure 1.4 represent Clean Architecture in a nutshell.Looking at this diagram, you can see that each layer is arranged in concentric circles with the Entity at the center.The arrows are drawn from the outside to the inside, which represents the dependency rule.Then, there is a collection of squares and arrows in the lower right corner of the figure, which represent the reversal of the processing flow and dependencies.We have already discussed dependency inversion in "Dependency Inversion Principle (DIP)" of "1.4 Design principles", and as you can see in this figure, dependency inversion is a very important concept.

Now, let's take a closer look at the elements that make up the Clean Architecture.

## Dependency rules

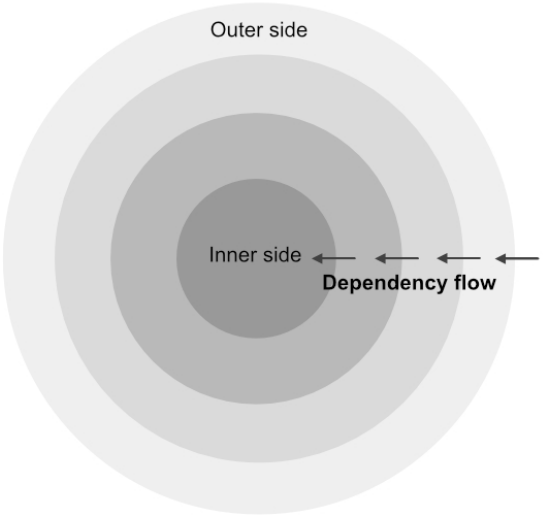


**Figure 1.5: concentric circles and dependency direction**

Figure 1.5 illustrates concentric circles and the direction of dependencies.There are business rules called Entities in the center, and layers are arranged in concentric circles around them.And this diagram contains important rules about dependencies.

> Source code dependencies must point only inward, toward higher-level policies.
>
> Robert C, Martin. "The Clean Architecture". Clean Architecture. Pearson Education.

To rephrase this rule, we can say that the inner element must not depend on the outer element.For example, a Use case can depend on an Entity, but the Entity cannot be the details of the Use case.In this way, the Entity is isolated and protected from outside changes.

In this concentric circle diagram (Figure 1.4 ), there is a "outside world of the application" outside the circle.And the outermost layers, such as the DB and UI, are connected to the outside world.In general, applications are driven by signals from the outside world,

such as user interactions, DB rewrites, and event subscriptions.In other words, the starting point of the process is the outer boundary of the concentric circle, which propagates toward the center of the circle.Then, the content processed inside the circle is transmitted to the outside again, and influences the outside world by presenting it to the user or rewriting it in the DB.To put it another way, the flow of processing starts from the outside of the concentric circles, goes through the inside, and returns to the outside again.The important thing here is that the flow of processing and dependencies are reversed.

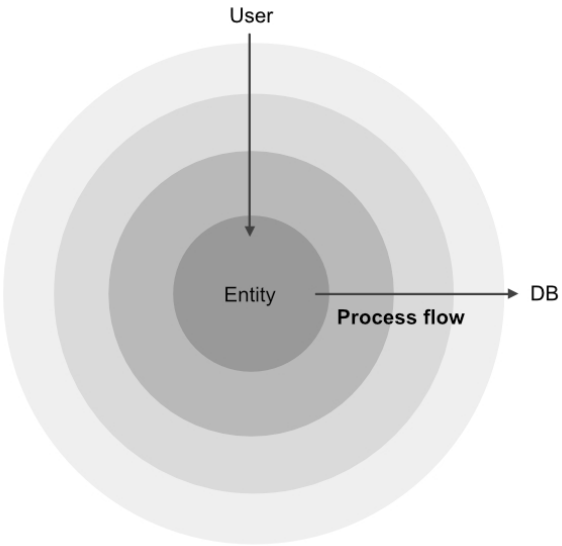


**Figure 1.6: process flow**

DIP and DI can control this reversal.DI will be explained in detail in Chapter 2"Dependency Injection" .

## Entity

> Entities encapsulate enterprise-wide Critical Business Rules.An entity can be an object with methods, or it can be a set of data structures and functions.It doesn’t matter so long as the entities can be used by many different applications in the enterprise.
>
> Robert C, Martin. "The Clean Architecture". Clean Architecture. Pearson Education.

Entities represent the rules of the business that the company is engaged in.Business rules are the core elements for applications, and they are configured to protect cores.Entities can be difficult to define clearly.The definition should be flexible, depending on the business the company is working on and the applications that make it possible.To think about what an Entity is, especially for an SPA such as a React application, we need to considerAPI servers and backend for frontend (BFF) are deeply involved.Entity handling in React applications is discussed in Chapter 3"Single Page Application" .

## Use case

> The software in the use cases layer contains application-specific business rules.
>
> Robert C, Martin. "The Clean Architecture". Clean Architecture. Pearson Education.

Whereas Entities dealt with the entire business of an enterprise, Use cases deal with the business rules specific to that application.The Use cases perform the necessary processing for the application while manipulating the Entities.In other words, Entities

abstract the processing across multiple applications, and more abstract rules than Use cases.

Again, how to handle Use cases in building an SPA is a matter of debate.

### Framework

In this book, we will focus on a specific view library called React and how to incorporate Clean Architecture ideas into React applications.However, in fact, the Clean Architecture is designed to be independent of any particular framework or library.In other words, it doesn't matter whether you use React or Vue.js for Clean Architecture.This is because the goal of this architecture is to separate frameworks and libraries from the application and make them easy to replace.Ideally, Clean Architecture should be applied to SPA, and the design should be able to switch between view frameworks (and libraries),but as a preliminary step, this book will focus on React while understanding that aspect of the Clean Architecture.

---

# 1.7. Main component

In general, there are several parts of the product code that do not satisfy the Dependency Inversion Principle (DIP).In other words, there are some components that are dependent on concrete components.Because concrete components are unstable and prone to change, components that depend on them are also affected by these changes.Clean Architecture uses DI (Dependency Injection) to reverse dependencies and eliminate dependencies on concrete components.However, in order to do DI, we need to depend on

concrete components.This is because DI is the process of indicating to an interface which concrete components to use.Thus, even if DI is applied to DIP violations, the dependency on concrete components will not disappear.Therefore, we will consider aggregating the dependencies of DI on concrete objects into one place. That part is called Main component.It is called this because this component often contains the `main` function.In other words, we use DI to fix multiple DIP violations in the product code to appropriate dependencies direction and aggregating dependencies on concrete components to Main component.

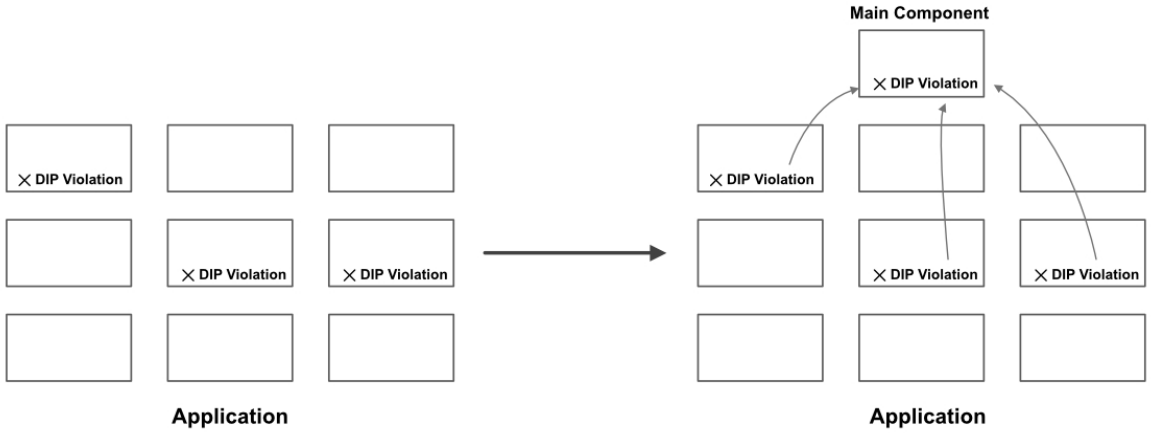


**Figure 1.7: Aggregate DIP violation into main component**

The dependency on concrete components cannot be completely removed from the product code.This is because you cannot use a concrete component without depending on it.Therefore, we can reduce the number of DIP violations by creating an element like Main component that takes care of the dependency on the concrete component.Of course, it is ideal to aggregate them in one place, but even if that is not the case, we need to aggregate them in as few places as possible.It is important to take into account the product's characteristics and the team's culture to design where Main components should be and what their roles should be.

## About the number and naming of layers

> The circles are intended to be schematic: You may find that you need more than just these four. There’s no rule that says you must always have just these four. However, the Dependency Rule always applies. Source code dependencies always point inward.
>
> Robert C, Martin. "The Clean Architecture". Clean Architecture. Pearson Education.

As Robert C. Martin notes, it is not always right to obediently follow what is written in these concentric circles.The most important thing is the dependency rule, not the number of concentric layers or their naming.There is no mention of layer naming in "Clean Architecture".On the other hand, naming your own unnecessarily would mean throwing away the benefit of following a particular architecture.This is because one of the advantages of following methods such as DDD or Clean Architecture is that developers who already know those methods for a particular naming can understand the architecture of the application at a glance.For example, if names such as `Entity` , `Use case` , `Repository` , etc. are used, you can guess what role the module will play without looking at the code details.Thus, even though we are adopting the Clean Architecture, it can be confusing to give it a different name than the naming of `Entity` or `Use case` .However, SPA and React have a naming culture that has been used traditionally as well.For example, `Store` , `Dispatcher` , `Reducer` , etc. used in Flux.How to map these naming cultures to the layered structure of the Clean Architecture is a matter of debate.This will be key to implementing Clean Architecture in React applications.These correspondences will be discussed a bit in Chapter 3"Single Page Application" .

# 1.8. Summary

In this chapter, we explained what Clean Architecture aims to achieve and what it specifies.Clean Architecture is based on design principles that have been cultivated throughout the history of software development.A deep understanding of each of these principles is important for applying Clean Architecture to real-world applications.In addition, we'll be breaking the principles and reinterpreting them in a practical way in applying Clean Architecture to React applications.With this in mind, the first thing to do is to familiarize yourself with Clean Architecture and understand its goals and principles.If you have any doubts in practicing Clean Architecture, reading this chapter and going back to the principles may help you to see things clearly.

# Chapter 2. Dependency Injection

Clean Architecture deals with concentric architecture with the business domain at the core, but One of the key principles is the dependency constraint, which states that we should only depend on the inside from the outside. However, in a straightforward implementation, the processing flow and dependencies would be in the same direction. Of course, the Clean Architecture also requires processing from the inside to the outside, so a straightforward implementation will result in dependencies from the inside to the outside. In other words, in order to observe the dependency principle, the flow of processing and the direction of dependency must be reversed. The technique for solving this problem is called dependency inversion. As the name suggests, this technique is used to satisfy the Dependency Inversion Principle (DIP). So how do we reverse the dependency? In this chapter, we introduce a technique called Dependency Injection (DI), which is commonly used to reverse dependencies.

## 2.1. DI library for TypeScript

Dependency Injection is a technique that has been used for a long time in statically typed languages such as Java.You can build your own mechanism to realize DI, or you can use existing DI libraries (DI frameworks).For example, in Java, there are many DI libraries such as Spring Framework and google/guice.These DI libraries allow you to use DI for developing various applications.

How about in the context of JavaScript and TypeScript?It may not be common to use DI in web frontend development.Nevertheless, there are DI libraries available for JavaScript and TypeScript.For example, AngularJS and Angular have a DI framework built in.There are also libraries that provide DI functionality on its own, such as InversifyJS*1 and TSyringe*2 .Both InversifyJS and TSyringe are DI libraries for use with TypeScript.InversifyJS is a library with over 5000 Github stars and is used in over 18,000 repositories.On the other hand, there doesn't seem to be much active development going on these days.TSyringe is a DI library that is mainly developed by Microsoft. As of September 2020, it has about 1,400 stars on Github, and it seems to be under continuous development.

In this chapter, we will first introduce a simple method to perform DI without using libraries.This simplified method uses a simple example to show what DI is trying to accomplish and what problems it has.We will then explain how TSyringe can be used to solve these problems.

[*1] http://inversify.io/

[*2] https://github.com/microsoft/tsyringe

---

# 2.2. Simple DI without libraries

In this section, we will use a simple example to illustrate DI without using libraries.The goal is to demonstrate an understanding of the overview of DI and the issues involved in DI.However, as will be

explained later, there are some practical problems with the method presented in this section.In the next section, we will confirm that these problems are solved by DI using the library.

In this section, we will deal with a simple subject as follows.

List 2.1: Document.ts

```
 1: import Printer from './Printer';
 2:
 3: class Document {
 4:    constructor(private content: string) {}
 5:
 6:    output() {
 7:      const printer = new Printer();
 8:      printer.print(this.content);
 9:    }
10: }
```

List 2.2: Printer.ts

```
1: export default class Printer {
2:    constructor() {}
3:
4:    print(content: string) {
5:      console.log('Print:', content);
6:    }
7: }
```

`Document` will output its own content using the `output` method.In the code above, the `output` method uses the class `Printer` internally.`Printer` prints out the received string by the `print` method.However, in order to simplify the implementation, we only use `console.log` for the output.The codes to use `Document` are as follows.

List 2.3: main.ts

```
1: import Document from './Document';
2:
3: const document = new Document('this is
sample text');
4: document.output() // "Print: this is sample
text"
```

Looking at the dependencies, we see a dependency from `Document` to `Printer` .Now let's assume that `Email` is added as an output of `Document` .An example codes for this would look like the following

List 2.4: Document.ts

```
 1: import Email from './Email';
 2: import Printer from './Printer';
 3:
 4: export default class Document {
 5:    constructor(private content: string,
private method: Printer | Email) {}
 6:
 7:    output() {
 8:      if (this.method instanceof Printer) {
 9:        this.method.print(this.content);
10:      } else {
11:        this.method.send(this.content);
12:      }
13:    }
14: }
```

List 2.5: Email.ts

```
1: export default class Email {
2:    constructor() {}
```

```
3:
4:    send(content: string) {
5:      console.log('Email:', content);
6:    }
7: }
```

New `Email` has been added, and `Document` now receives the output method in the constructor.The dependency in this implementation is still the same as before, from `Document` to `Printer` .In addition, `Document` now also depends on `Email` .The code for the user side looks like this.

List 2.6: main.ts

```
1: import Document from './Document';
2: import Printer from './Printer';
3: import Email from './Email';
4:
5: const printerDocument = new Document('this
is sample text', new Printer());
6: printerDocument.output(); // "Print: this
is sample text"
7:
8: const emailDocument = new Document('this is
sample text', new Email());
9: emailDocument.output(); // "Email: this is
sample text"
```

As you may have already noticed, this implementation violates the DIP.This is because `Document` depends on the concrete implementations `Printer` and `Email` .This means that if `Printer` or `Email` is changed, `Document` will also be affected.In a simple example, if you rename the `print` method of `Printer` , you will need to modify the `output` method of `Document` too.Also, if you want to add more output methods for `Document` , as you did for

`Email` , you need to modify `Document` .This is a violation of the Open-Closed Principle (OCP): "open to extensions".To solve the above problems, we will use DI to reverse the dependencies.Reverse the dependency in the following steps.

1. Create `Document` interface on `DocumentOutputMethod` side
2. `Printer` and `Email` implement the `DocumentOutputMethod` interface
3. `Document` implements the process using the interface
4. Pass an instance of the class that implements the `DocumentOutputMethod` interface to the constructor of `Document` in the user side codes

This is called Dependency Injection (DI) because the instance to be depended on in step 4 is injected from the outside.Let's take a look at an example implementation.

List 2.7: Document.ts

```
 1: export interface DocumentOutputMethod {
 2:     output: (content: string) => void;
 3: }
 4:
 5: export default class Document {
 6:     constructor(private content: string,
private method: DocumentOutputMethod) {}
 7:
 8:     output() {
 9:       this.method.output(this.content);
10:     }
11: }
```

Added the `DocumentOutputMethod` interface under the management of `Document` .And `Document` uses this `DocumentOutputMethod` to implement the `output` method.By doing this, you can see that it no longer depends on specific implementations such as `Printer` or `Email` .

List 2.8: Printer.ts

```
 1: import { DocumentOutputMethod } from
'./Document';
 2:
 3: export default class Printer implements
DocumentOutputMethod {
 4:     constructor() {}
 5:
 6:     print(content: string) {
 7:       console.log('Print:', content);
 8:     }
 9:
10:     output(content: string) {
11:       this.print(content);
12:     }
13: }
```

List 2.9: Email.ts

```
 1: import { DocumentOutputMethod } from
'./Document';
 2:
 3: export default class Email implements
DocumentOutputMethod {
 4:     constructor() {}
 5:
 6:     send(content: string) {
 7:       console.log('Email:', content);
 8:     }
 9:
```

```
10:     output(content: string) {
11:       this.send(content);
12:     }
13: }
```

`Printer` and `Email` implement the `DocumentOutputMethod` interface.Added `output` method to satisfy the `DocumentOutputMethod` interface.This means that `Printer` and `Email` now depend on `Document` .The above fix reversed the dependency.

List 2.10: main.ts

```
 1: import Document from './Document';
 2: import Printer from './Printer';
 3: import Email from './Email';
 4:
 5: const printerDocument = new Document(
 6:     'this is sample text',
 7:     new Printer()
 8: );
 9: printerDocument.output();
10:
11: const emailDocument = new Document('this
is sample text', new Email());
12: emailDocument.output()
```

Finally, in the code of the user side, the constructor of `Document` is passed an instance of a class that implements the `DocumentOutputMethod` interface.Now you can do DI without using the library.

## Problems with this method

Actually, there is a problem with this method.That is, when you instantiate `Document` , you need to instantiate and pass in `Printer` and `Email` .This is another way of saying that any code that uses `Document` will always depend on `Printer` and `Email` .In the above example, `main.ts` is affected by this, butIf you use `Document` in multiple locations, each location will depend on `Printer` and `Email` .After removing the dependency on the concrete implementation from `Document` , this just shifts the dependency elsewhere.To solve this problem, the dependency on concrete code should be gathered in as few places as possible, as described in "1.7 Main component" .

So how do we aggregate our dependency on concrete code?This can be solved by using an object called a DI container, which has a correspondence table of dependencies.The DI container holds a list of correspondences between the class to be injected (`Document` in this case) and the class to inject (`Printer` or `Email` ).When instantiating the class to be injected, it automatically selects the class to inject from the correspondence table of the DI container, instantiates it, and passes it on.Then, the description of the correspondence of the DI containers is integrated into the main component or configuration files.In this way, the class to be injected (`Document` ) can be used in various places while consolidating the dependencies on concrete code.

It is of course possible to implement a DI container without any libraries.However, there are a number of technical issues that need to be resolved, and it is quite difficult to implement them on your own.This is a brief bullet list of issues that need to be resolved.If you are interested, please check out the details.

- Need to use decorator
- Metadata needs to be retrieved by reflection
- TypeScript interface information is removed at compile time and cannot be mapped in the DI container.

These points have been resolved in the existing DI library.In the next section, we will show an example of a DI that solves this problem by using TSyringe.

---

# 2.3. TSyringe

TSyringe is a DI library developed under the leadership of Microsoft.As the name suggests, TSyringe allows you to introduce DI into your TypeScript development.

## TypeScript Decorators

TSyringe uses an experimental feature in TypeScript called Decorators.Decorators are in the Stage 2 Draft stage of JavaScript as of March 2021.As such, there is room for change as a specification.Similarly, TypeScript has been implemented it as an experimental feature, and its specifications are subject to change in the future.

So why does TSyringe use Decorators?This is because the nature of the DI library is such that it works well with decorators (as a general language feature).The DI library needs to know in some way the class to which it is injecting dependencies.Before decorators were used, the injection target was set by a configuration file.In this way, it is not possible to know what the DI settings are in the file where the class is declared.There is also the cost of writing and managing configuration files.By using decorators, these problems can be solved.The following is an example of using TypeScript Decorators (not related to TSyringe).

List 2.11: Example of using decorators

```
 1: function methodDecorator(target: any,
props: string, descriptor: PropertyDescriptor)
{
 2:     console.log('This is method decorator
f()');
 3: }
 4:
 5: function classDecorator(constructor:
Function) {
 6:     console.log('This is class decorator
f()');
 7: }
 8:
 9: @classDecorator
10: class SampleClass {
11:       @methodDecorator
12:       hoge() {
13:           return null;
14:       }
15: }
```

In order to use Decorators in TypeScript, you need to add the following setting in the compiler options.

List 2.12: tsconfig.js

```
1:    {
2:      "compilerOptions": {
3:        "experimentalDecorators": true
4:      }
5:    }
```

## Installation and initial setup

This section describes the setup for using TSyringe.The information here is based on official documentation.*3 First, install the package.

[*3] https://github.com/microsoft/tsyringe/blob/master/README.md

Install the package `tsyringe` by

```
$ npm install --save tsyringe
```

or,

```
$ yarn add tsyringe
```

TSyringe uses an experimental feature called Decorators in TypeScript, so we will rewrite `tsconfig.js` as follows

List 2.13: tsconf.js

```
{
    "compilerOptions": {
      "experimentalDecorators": true,
      "emitDecoratorMetadata": true
    }
}
```

We can use Decorators by enabling `experimentalDecorators` .You can also enable `emitDecoratorMetadata` to allow the decorator to handle a lot of metadata.To enable the Reflect API to handle metadata, you need to select one of the following libraries to use as a polyfill.

- reflect-metadata
- core-js (core-js/es7/reflect)
- reflection

In this example, we install `reflect-metadata` as shown below,

```
$ npm install --save reflect-metadata
```

or,

```
$ yarn add reflect-metadata
```

`reflect-metadata` needs to be `import` only once before using DI.It is a good idea to do `import` at the top level as much as possible.

List 2.14: main.ts

```
1: import "reflect-metadata";
```

Now you are ready to use TSyringe.Next, let's look at how to actually do DI.

## Define the class where you want to do DI

TSyringe provides a simple API for doing DI.The implementation code differs depending on whether `interface` is used for the type of the object to be injected or not.Here is an example of how to use `interface` .For more information, please refer to the official documentation.*4

[*4] https://github.com/microsoft/tsyringe

First, define the type to be injected with `interface` .

List 2.15: Interface of API to get user information

```
1: // apiInterface.ts
2: export interface User {
3:     id: number;
4:     name: string;
```

```
5: }
6:
7: export interface UserApiInterface {
8:    fetchUser: (args: { id: number }) =>
User;
9: }
```

`UserApiInterface` defines an API for retrieving user information.It is assumed that the class that implements this interface includes processes such as communicating with the API server and connecting directly to the DB.In the following, as a mock, we will implement a class that receives a user ID and returns static information.

List 2.16: Implementation of API to get user's information

```
 1: // apiImpl.ts
 2: import { UserApiInterface, User } from
'./apiInterface';
 3:
 4: export class UserApiImpl implements
UserApiInterface {
 5:    fetchUser({ id }: { id: number }): User
{
 6:      return {
 7:        id,
 8:        name: `This is fetchUse of UserApi:
${id}`.
 9:      };
10:    }
11: }
```

Then, define the class in which `UserApiInterface` should be injected.

List 2.17: definition of DI

```
 1: // api.ts
 2: import { injectable, inject } from
'tsyringe';
 3:
 4: import { UserApiInterface } from
'./apiInterface';
 5:
 6: @injectable()
 7: export class UserApi {
 8:    constructor(
 9:      @inject('UserApiInterface') private
api: UserApiInterface
10:    ) {}
11:
12:    fetchUser(args: { id: number }) {
13:      return this.api.fetchUser(args);
14:    }
15: }
```

`UserApi` receives an object of type `UserApiInterface` in `constructor` as an argument and keeps it as a private member.Then, in `UserApi.fetchUser()` , it calls `api.fetchUser()` , which is implemented in `api` .In this way, `UserApi` can change its behavior depending on the object passed in `constructor` .By writing `@inject('UserApiInterface')` in `constructor` , we pass the interface metadata to the DI library side.

## Injecting an object

Now, inject the object into `UserApi` .

List 2.18: object injection

```
 1: // main.ts
 2: import { container } from 'tsyringe';
 3:
  4: import { UseApiImpl } from './apiImpl';
 5: import { UserApi } from './api';
 6:
 7: container.register('UserApiInterface', {
 8:    useClass: UserApiImpl
 9: });
10:
11: const userApi =
container.resolve(UserApi);
```

`container` is TSyringe's DI container, which holds the information to perform DI.This `container` 's `register()` method sets the object to be injected.In this example, we are configuring `UserApiInterface` to inject `UserApiImpl` .And finally, `container.resolve(UseApi)` instantiates `UserApi` .At this time, `container` instantiates `UserApiImpl` and passes it to `UserApi` .

In other words, TSyringe is a mechanism to create instances while resolving dependencies (which registered by `register()` ) by `resolve()` .

This is the basic usage of TSyringe.This DI mechanism can be used to reverse the dependency.

# 2.4. Dependency Inversion with DI Library

In the previous section, we presented an example of DI using TSyringe.As many readers may have noticed from the implementation examples, using a DI library like TSyringe does not automatically reverse the dependencies.The DI library does this by externally selecting and injecting the objects that a class depends on.Therefore, it is possible to reverse the dependency by using it properly, but on the other hand, it is not possible to change the direction of the dependency if it is not used properly.This section describes how to use the DI library to reverse the dependencies.

## Design modules

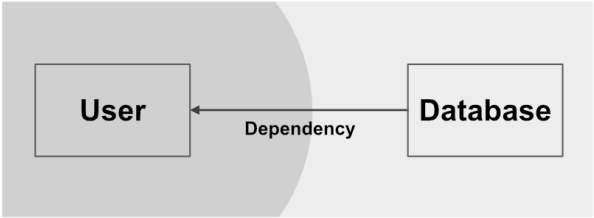


**Figure 2.1: target inter-module dependencies**

First, let's design modules that will be the subject of our example.In this section, we will consider the `User` model, which is the domain model, and `Database` , which is responsible for persisting the model.`User` corresponds to the business logic and is located at the center of the concentric circles diagram.And since `Dataset` is a concrete module, it is located outside the concentric diagram.Therefore, in accordance with Clean Architecture, the dependency goes from `Database` to `User` .This is represented in the diagram as Figure 2.1 .`Database` can know about `User` , but

conversely, `User` cannot know about `Database` .For the sake of illustration, let's assume that one file corresponds to one module, and create `user.ts` and `database.ts` .

First, let's define each one without considering the correct dependencies.

List 2.19: user.ts

```
 1: import Database from './database';
 2:
 3: export default class User {
 4:    private _id: number | null;
 5:
 6:    private _name: string;
 7:
 8:    constructor(private database: Database)
{
 9:      this._id = null;
10:      this._name = '';
11:    }
12:
13:    set id(value: number) {
14:      this._id = value;
15:    }
16:
17:    set name(value: string) {
18:      this._name = value;
19:    }
20:
21:    get name() {
22:      return this._name;
23:    }
24:
25:    save() {
26:      if (typeof this._id === 'number') {
27:        this.database.save(this);
28:      }
```

```
29:     }
30: }
```

`User` can be set to `id` and `name` .It persists itself via `Database` by `User.save()` .

List 2.20: database.ts

```
1: import User from './user';
2:
3: export default class Database {
4:     save(user: User) {
5:       console.log(`${user.name} has been
saved`);
6:     }
7: }
```

`Database` will receive the `User` model and persist it.However, for the sake of simplicity, we will only display the received `User` 's `name` in the console.

Let's take a look at the dependency between `User` and `Database` defined so far (Figure 2.2 ).`Database` is `import` the type information of `User` .`User` also uses `Database` as `import` , butThis is because `User` uses `Database` for persistence.If you look at these dependencies, you can see that they violate the dependency rules.

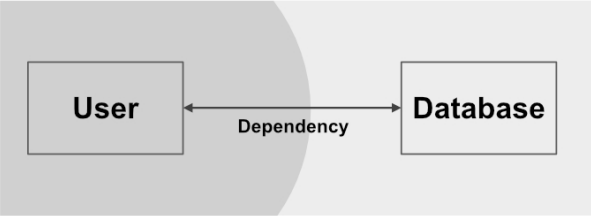


**Figure 2.2: Dependencies between modules at the moment**

Now, let's fix the dependency to the correct orientation.

## Reversing dependencies between modules

First, define `DatabaseInterface` in `user.ts` .

List 2.21: user.ts

```
 1: export default class User {
 2:     private _id: number | null;
 3:
 4:     private _name: string;
 5:
  6:     constructor(private database:
DatabaseInterface) {
 7:       this._id = null;
 8:       this._name = '';
 9:     }
10:
11:     set id(value: number) {
12:       this._id = value;
13:     }
14:
15:     set name(value: string) {
16:       this._name = value;
17:     }
18:
19:     get name() {
20:       return this._name;
21:     }
22:
23:     save() {
24:       if (typeof this._id === 'number') {
25:         this.database.save(this);
26:       }
27:     }
```

```
28: }
29:
30: export interface DatabaseInterface {
31:     save: (user: User) => void;
32: }
```

Change the `constructor` argument of `User` to `DatabaseInterface` .This will remove `import` from `user.ts` to `database.ts` .`User` is no longer dependent on `Database` .Next, modify `Database` so that it implements `DatabaseInterface` .

List 2.22: database.ts

```
1: import User, { DatabaseInterface } from
'./user';
2:
3: export default class Database implements
DatabaseInterface {
4:     save(user: User) {
5:       console.log(`${user.name} has been
saved`);
6:     }
7: }
```

`Database` is importing `User` , which turns out to be fine according to the dependency rules we were aiming for.Now we have satisfied the dependency rule.The last step is to configure TSyringe.

List 2.23: user.ts

```
 1: import { injectable, inject } from
'tsyringe';
 2:
 3: @injectable()
 4: export default class User {
 5:     private _id: number | null = null;
```

```
 6:
 7:    private _name: string = '';
 8:
 9:     constructor(
10:      @inject('DatabaseInterface')
11:      private database: DatabaseInterface
12:    ) {}
13:
14:    set id(value: number) {
15:      this._id = value;
16:    }
17:
18:    set name(value: string) {
19:      this._name = value;
20:    }
21:
22:    get name() {
23:      return this._name;
24:    }
25:
26:    save() {
27:      if (typeof this._id === 'number') {
28:        this.database.save(this);
29:      }
30:    }
31: }
32:
33: export interface DatabaseInterface {
34:    save: (user: User) => void;
35: }
```

Added `User` with `@injectable()` and added `database` argument of `constructor` with the decorator `@inject('DatabaseInterface')` .Now you can inject `Database` into `User` by `container` .Let's take a look at the code that uses these modules.

List 2.24: app.ts

```
 1: import { container } from 'tsyringe';
 2: import User from './di/user';
 3: import Database from './di/database';
 4:
 5: container.register('DatabaseInterface', {
 6:     useClass: Database
 7: });
 8:
 9: export const user =
container.resolve(User);
10:
11: user.id = 0;
12: user.name = 'test user';
13: user.save(); // 'test user saved' will be
displayed in the console
```

This reversed the dependency and helped us to follow the rules.The important thing here is to define `DatabaseInterface` , on which `User` originally depends, in the `User` module.In this way, `User` can manage `DatabaseInterface` within the scope of its own responsibilities without directly relying on `Database` .In other words, if you define `DatabaseInterface` in the `Database` module, it will lose its meaning.

Now we know that with proper use of interfaces and DI, we can reverse the dependencies between modules.

## Aggregating dependencies on concrete code

As described in "Problems with this method" in the previous section, DI without libraries has a problem that it cannot aggregate dependencies on concrete code.In the example in this section using TSyringe, we can aggregate `container.register()` into

`app.ts` .And by using `container.resolve(User)` , the code on the side using `User` no longer depends on `Database` .In this way, we can see that the problem that the simple DI without the library has been solved.

---

# 2.5. Summary

In this chapter, we introduced TSyringe, a lightweight DI library for TypeScript.We also showed how to use interfaces and DI to reverse dependencies.In the following chapters, we will show how this technique can be applied to React applications in practice.

# Chapter 3. Single Page Application

It is still not common to adopt Clean Architecture in web frontend development. Originally, Clean Architecture has been a traditional focus for server applications and native applications. This is probably due to the fact that it has been needed for designing complex applications. Traditional development using JavaScript is limited to adding lightweight behavior to server-side rendered HTML and dynamic screen updates using Ajax. However, the responsibilities of the web frontend have increased in recent years as browsers have become more sophisticated and JavaScript execution environments have become faster. In particular, the spread of the single page application (SPA) concept has led to the construction of rich applications. An SPA requires more complex display logic on the part of the web application. It can be said that the logic that used to be in the server application is now being transferred to the web frontend. And as applications become more complex, they require more sophisticated designs. In other words, design theories such as Clean Architecture are becoming more and more important as SPA is adopted in web frontend. In this chapter, we will consider SPA from a new perspective by looking at it from the perspective of Clean Architecture.

# 3.1. Difference from the Original Book

In this book, we will attempt to apply the Clean Architecture to SPA.However, what is written in the original book "Clean Architecture" cannot be directly applied to SPA.This is because there are some differences between the context in which the original book was written and the current situation in which the demand for SPA has grown.

## SPA as View

Clean Architecture is essentially a method for designing an entire product, and in this context, SPA is equivalent to View.In the original Clean Architecture, the View is the outermost part of the architecture.This is because the View is an unstable layer that is prone to change, and we want to keep it as far away from the domain model as possible.

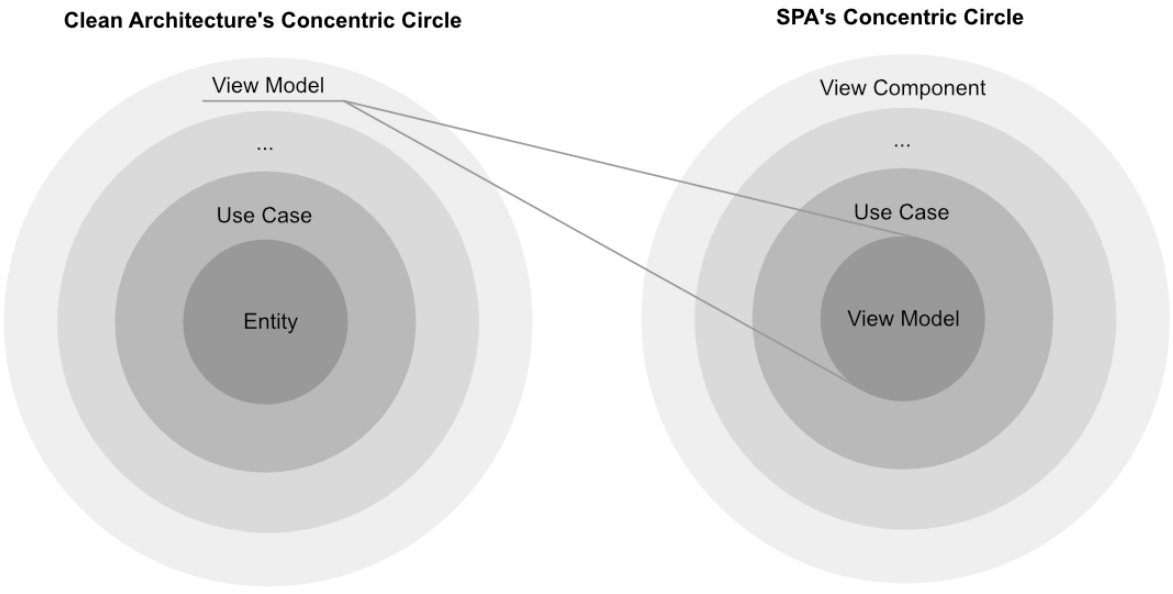


**Figure 3.1: Relationship between Clean Architecture concentric circles and SPA concentric circles**

On the other hand, this book aims to bring a Clean Architecture perspective to React applications.This is due to the fact that web applications are increasingly being implemented as SPA as web technology and browser functionality develops, and that the number of SPA implementations is increasing.This is also the reason why complex designs are increasingly required for their SPA.With SPA, you are building a single application using only frontend technologies.In other words, the model exists inside the SPA, and the Controller and Use case equivalents exist around it.It can be said that the designs required for SPA have become large enough and complex enough to make room for design techniques such as Clean Architecture.And as you can see from this background, applying Clean Architecture to SPA is not obvious and requires ingenuity, trial and error.

## About Deploying

In addition, the original book, Clean Architecture, had in mind large-scale applications in languages that require compilation, such as Java.For this reason, it is important to keep the impact of modifying the code to the minimum necessary components (i.e., units of deployment).This is because if many components are affected, you will need to compile them all and also deploy them.The principles of Clean Architecture are heavily influenced by this background.

On the other hand, in considering SPA, the impact on deployment is different from the situation in the original one.For example, given the mainstream TypeScript (or JavaScript) ecosystem for building SPA, it doesn't cost much to transpile the entire application at deployment time.In addition, although not limited to SPA, in recent years, ideas such as CI (Continuous Integration) and CD (Continuous Delivery) have become popular, and are becoming more and more important.There are probably many sites that have adopted this approach.In an environment where products are released continuously and automatically through CI/CD, the impact

on costs will not be as large as before, even if the number of affected components increases slightly.

Thus, in applying Clean Architecture to SPA, we need to consider a slightly different situation from the background described in the original book.Rather than adopting all the principles of Clean Architecture as they are, it is necessary to determine the essential effects and make a selection.

---

# 3.2. Framework

As mentioned in "Framework" of "1.6 Clean Architecture", Clean Architecture is framework-independent.In a concentric circle architecture diagram, the framework is located at the outermost part of the circle.This is due to the fact that frameworks are more changeable than business logic.In "Clean Architecture", Uncle Bob describes the framework as follows

> In effect, the author is asking you to marry the framework—to make a huge,long-term commitment to that framework. And yet,under no circumstances will the author make a corresponding commitment to you.It’s a one-directional marriage.You take on all the risk and burden; the framework author takes on nothing at all.
>
> Robert C, Martin. "Frameworks Are Details". Clean Architecture. Pearson Education.

This is a value that has been derived from Uncle Bob's experience.On the other hand, in recent years, it has become more and more active as an OSS, and many committers and maintainers

are involved in a single framework.This makes it possible for anyone to participate in the development and design of frameworks.Of course, there is no denying that there are companies and organizations that are the main developers of OSS and that they have a great deal of control over it.When it comes to React, Facebook is the main developer and the largest user, so it is natural that their demands would be reflected in many ways.Nevertheless, I believe that the risk of being "married" to a framework has been lowered compared to the past.

By the way, what are the advantages of designing independently from a framework?By making the application independent of the framework, it delays technical decision making and gives the application more freedom.In addition, if the framework needs to be replaced, it can be done so at a low cost.In terms of SPA, you would build your application logic independently from frameworks and libraries such as React, Vue.js, and Angular.How to separate the logic from the framework is the key to applying Clean Architecture to SPA.

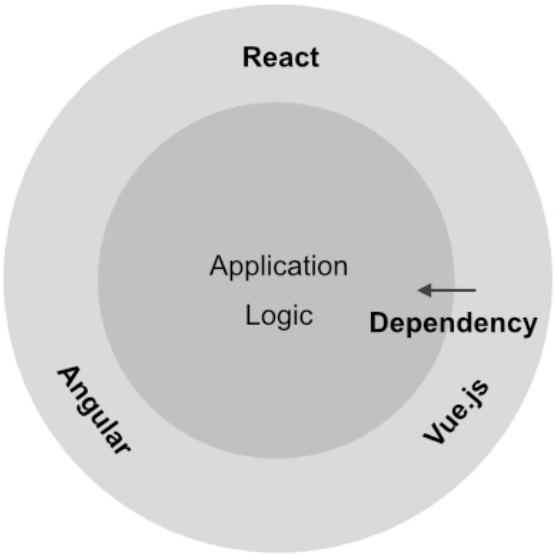


**Figure 3.2: Separation of logic and framework**

This book assumes React, and unfortunately I couldn't mention the framework separation, but I would like to try it in the future if I have the chance.

---

# 3.3. Relationship between SPA and Concentric Diagrams

In the Clean Architecture approach, the UI is the outermost part of the concentric circle diagram.This is because the UI is a module that is prone to change.Therefore, it is placed away from the Entity that is the business domain.How should we view this in terms of SPA?As a concrete example, let's consider the following two cases

1. the SPA backend is a managed service like Firebase
2. the SPA backend is the API server

From a design perspective, what is the difference between 1 and 2?When you look at it from the perspective of an SPA, there is a difference in whether the communication destination is Firebase or an API server, and although the communication interface is different, it will not have a significant impact on the design.On the other hand, the situation is a little different when you look at the product as a whole.In the first case, the SPA is directly interacting with Firebase, so all business logic is implemented on the SPA side.In other words, the architecture diagram for the entire product and the SPA-centric diagram will be the same.

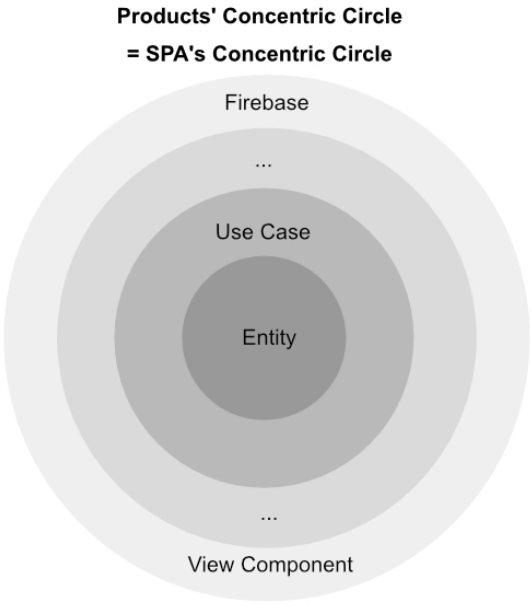


**Figure 3.3: Example when the concentric circles of the product and the concentric circles of the SPA are the same**

On the other hand, what about 2.In the case of 2, the SPA will be located outside of the concentric circles in terms of the overall product relationship.The API server holds the business logic and does the important processing.*1 The SPA receives and displays data that has been processed by the API server.On the other hand, you know that SPA is not just about displaying the received data as it is.Data needs to be handled properly within the SPA as well.In other words, while the SPA is located outside the concentric circles when viewed as a whole product, another concentric circle diagram can be drawn when viewed with the SPA as the center.

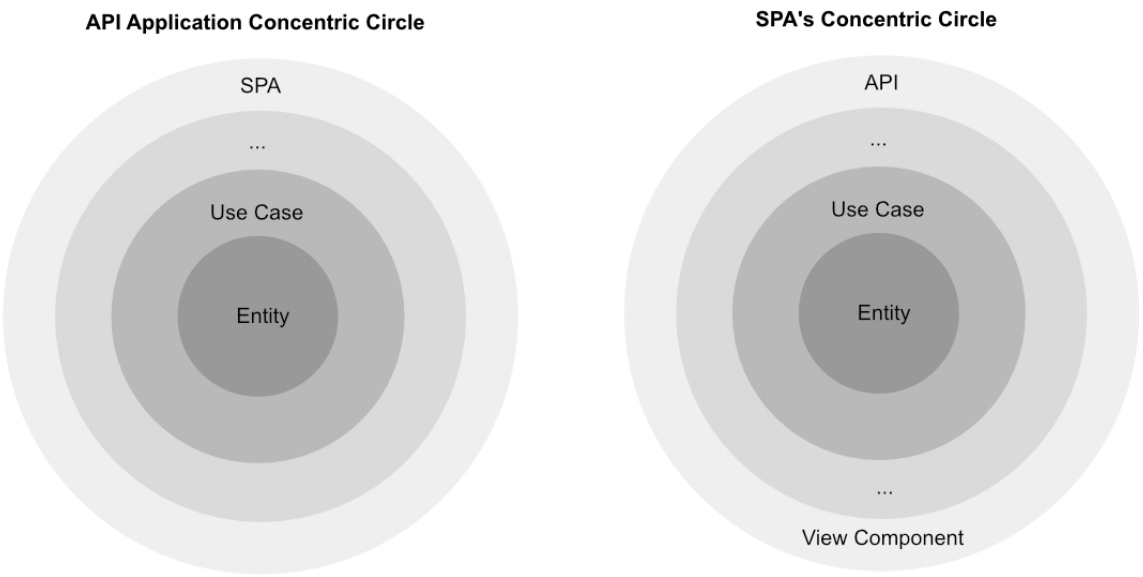


**Figure 3.4: Example of different product concentric circles and SPA concentric circles**

Thus, by recursively applying the structure of Clean Architecture, we can consider concentric circles around the SPA.In the following, we will refer to the concentric circle diagram that captures the entire product as the "product concentric circle diagram" and the concentric circle diagram centered on the SPA as the "SPA concentric circle diagram".Here, how can we map the respective concepts of the concentric circle diagram that captures the original product as a whole and the concentric circle diagram centered on the SPA?In the next section, we will look at the mapping.

[*1] It depends on the design, but as long as the API server exists, it is natural to do so

---

# 3.4. Entity and View models

The Entity is located at the center of the product concentric diagram.As explained in "Entity" of "1.6 Clean Architecture", an

Entity represents the business rules that a company is working on.This means that there is only one Entity for that company or product.So what is at the center of the SPA concentric circles diagram?This is where the concept of "View model" comes into play.The View model is a concept introduced in "Clean Architecture" and is the transformation of an Entity into a display.It is good to think of this View model as being at the center of the SPA.In other words, it can be said that the logic (Presenter) and View models for UI display are cut out from the product concentric circle diagram.

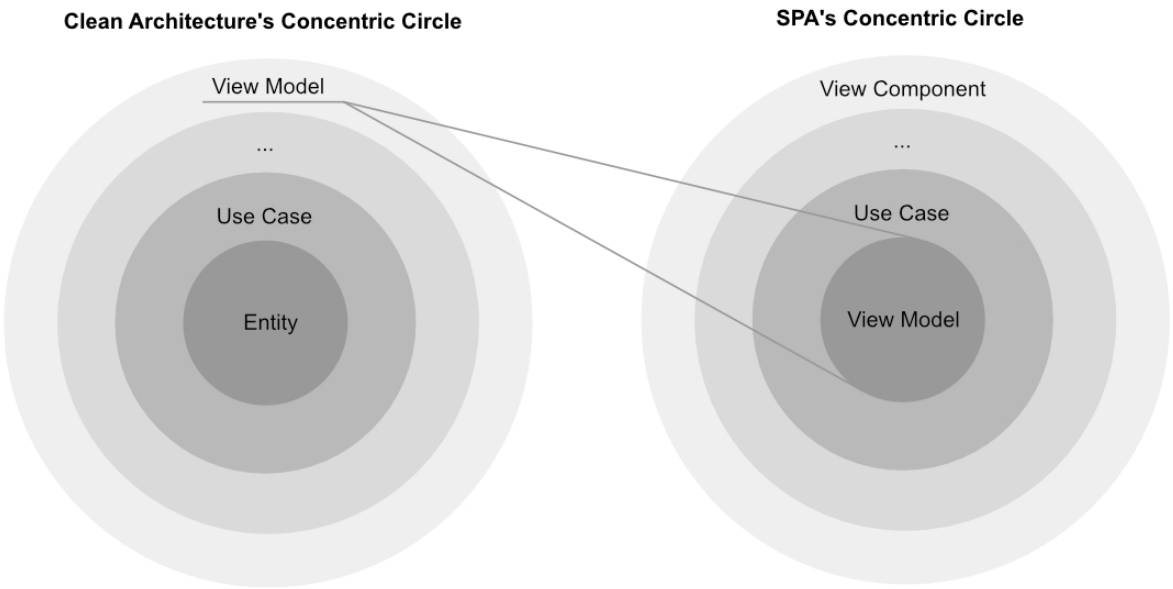


**Figure 3.5: Relationship between Clean Architecture concentric circles and SPA concentric circles (again)**

As you can see, the View model is located at the center of the SPA concentric diagram, and it carries the data and logic necessary to display it in the SPA.

# 3.5. Use Case

Use case is located outside one of the entities in the product concentric circles diagram.

> The software in the use cases layer contains application-specific business rules.It encapsulates and implements all of the use cases of the system.These use cases orchestrate the flow of data to and from the entities,and direct those entities to use their Critical Business Rules to achieve the goals of the use case.
>
> Robert C, Martin. "The Clean Architecture". Clean Architecture. Pearson Education.

In this way, Use cases imply application-specific business rules.It performs the necessary processing for display while controlling the Entity (or View model in the case of SPA).In SPA development, it is also possible to add and develop a layer of Use cases.By using the naming of Use cases in SPA development as well, the architecture will become clearer with Use cases as the common language.On the other hand, SPA has the concept of data flow, such as Flux and Redux.The Use cases overlap with these.

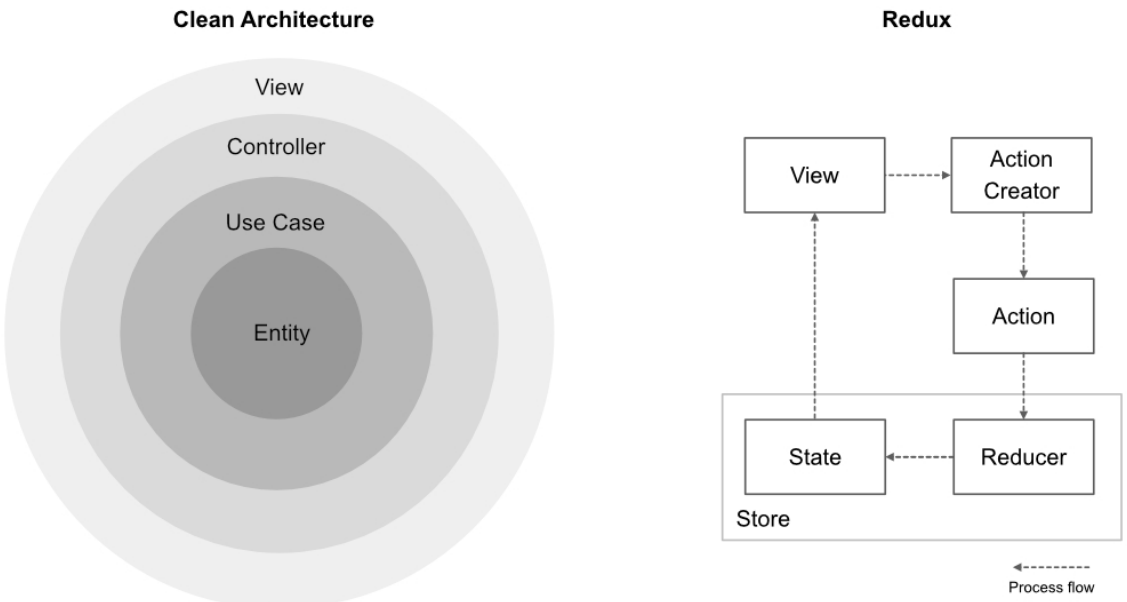


**Figure 3.6: Clean Architecture vs. Redux**

In order to apply Clean Architecture while adopting Flux and Redux, we need to understand the relationship between the two structures.In the next section, we discuss the structure of Redux and its relation to Clean Architecture, including Use cases.

---

# 3.6. Redux and Clean Architecture

In this section, we will focus on Redux, which is often used in SPA and native applications, and compare its structure with that of Clean Architecture.After introducing the basic structure of Redux, we will examine Redux from the perspective of Clean Architecture.And finally, we will discuss how to make Clean Architecture and Redux coexist.

## Review of Redux

Redux is a framework for state management based on Flux, but with more restrictions to make it easier to handle.Redux is made up of the following six components

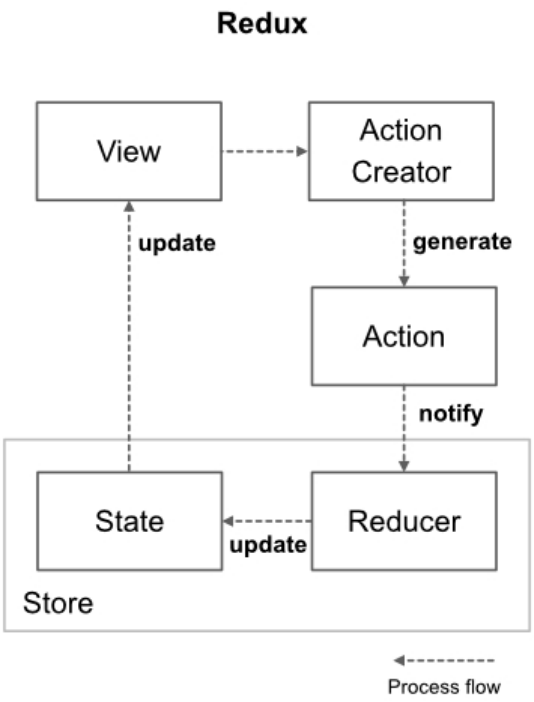


**Figure 3.7: Redux processing flow (overview)**

- View
- Action
- Action Creator
- Store
- State
- Reducer

And then there are the following three principles.*2

1. single source of truth
2. State is read-only
3. Changes are made with pure functions

There is a single read-only `State` . This is in accordance with Principle 1.And `Reducer` is the receiver for updating `State` .`State` and `Reducer` together are called `Store` .`Reducer` subscribes to `Action` , retrieves data from the received `Action` , and updates `State` .The fact that `Reducer` subscribes to `Action` is the "observer pattern" in the design pattern.By following Principle 2, `State` will be read-only. Therefore, `Reducer` will create a new `State` to update and replace `State` .This means that `State` should be immutable.

`Action Creator` is a function that creates `Action` . `Action` itself is just a data structure for conveying information.The role of `Action Creator` is to store and create the necessary information for `Action` .So how do we fire `Action` to tell `Reducer` ?Redux provides the `dispatch` function.By passing `Action` to the `dispatch` , `Action` will be passed to the `Reducer` subscribing it.

As stated in Principle 3, `Reducer` and `Action Creator` should be configured as pure functions with no state.By doing this, the state will be only `State` in the Redux framework, and no side effects will occur.By using immutable `State` and pure functions, there are no unintended state changes or side-effects, which makes it easier to reproduce the situation and write tests.

Nevertheless, there are often cases where you want to trigger side effects starting from `Action` .For example, you can use the information received from `Action` to communicate to the API server, and then issue `Action` again to update `State` after the communication is completed.The `Middleware` is used in such

cases. (I will skip the detailed explanation of `Middleware` ).`Middleware` is a mechanism to externally insert a process to subscribe to `Action` .

As you can see in Figure 3.7 , Redux is designed that data flow in one direction.In general, the larger the application, the more complex the data flow tends to become.Redux provides a unidirectional flow of data, and by constraining updates to always go through `Action` or `Reducer` .This makes it easier to understand what is going on in the application.

[*2] https://redux.js.org/introduction/three-principles/

## Redux and concentric circles

Let's take a look at the components of Redux and their dependencies.

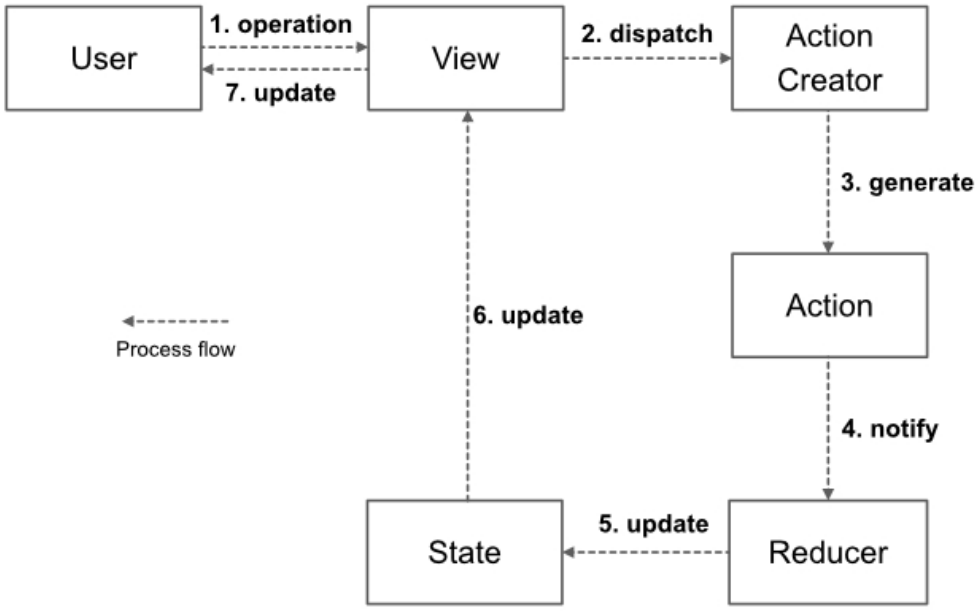


**Figure 3.8: Redux process flow (detail)**

The basic processing flow of Redux begins with user interaction with `View` .`View` corresponds to a view library such as React or View.js.When `View` receives an interaction, `Action Creator` creates the appropriate `Action` , and by `dispatch` ing it, `Action` is passed to `Reducer` .The `Reducer` that subscribed to the corresponding `Action` will process and update the `State` .Then, when `State` is updated, `View` will also be updated and the content displayed will change.This is the flow of the Redux process.

Next we will discuss dependencies.Considering the principles of Clean Architecture, the following dependencies are defined.Note that this dependency is not specified by Redux, but is just an example of "this dependency design may be appropriate based on the principles of Clean Architecture".

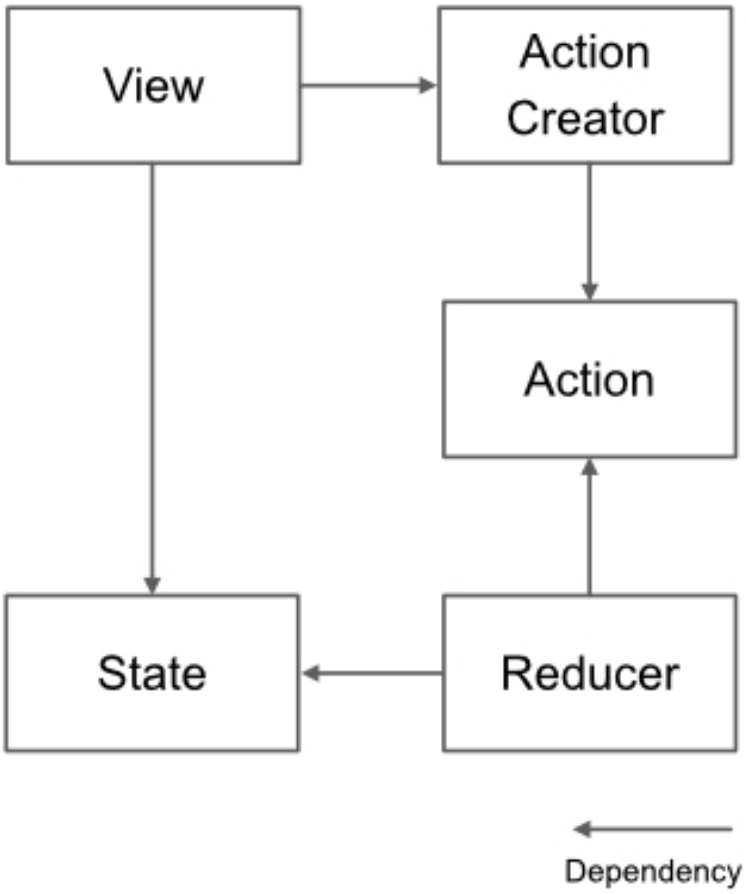


**Figure 3.9: Redux dependency**

This dependency can be put into words as follows.

1. `View` depends on `Action Creator` and `State`
2. `Action Creator` depends only on `Action`
3. `Action` is independent
4. `Reducer` depends on `Action` and `State`
5. `State` is independent

Let's look at the details of each.

## 1. View depends on Action Creator and State

`View` will execute `Action Creator` in response to user interaction.At this point, `View` doesn't need to know the contents of `Action` .`View` depends on `State` because it changes the display contents according to the contents of `State` .Also, there are no elements that depend on `View` . We can say that it is located at the outermost part of the concentric circle diagram.This is reasonable in view of the fact that `View` is prone to change.

## 2. Action Creator depends only on Action

`Action Creator` is responsible for filling `Action` with information.Therefore, `Action Creator` needs to know the contents of `Action` .Also, `Action Creator` is independent from the other elements.

## 3. Action is independent

`Action` does not depend on any other elements.It is an independent and change-resistant element. It will be located in the center of the concentric circle diagram.

## 4. Reducer depends on Action and State

`Reducer` subscribes to `Action` , retrieves information from it, and processes it.Therefore, `Reducer` needs to know the contents of `Action` .Also, since `Reducer` updates `State` , it must know the contents of `State` .

## 5. State is independent

`State` is independent from the other elements.Since `State` is an element that corresponds to `Entity` in the concentric circles, it is reasonable for it to be independent of the others.And it will be in the center on the concentric circles diagram.

Based on the above, let's fit the components of Redux into a concentric circle diagram.

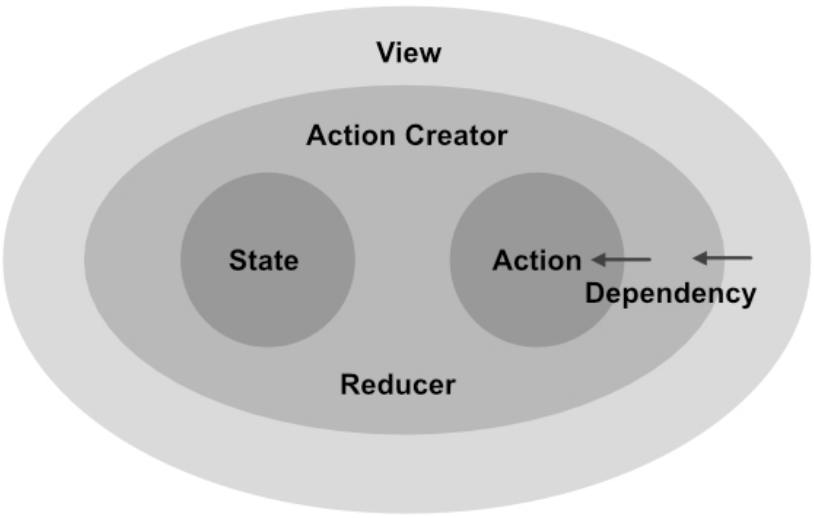


**Figure 3.10: Redux concentric circles**

In this way, we were able to place `State` in the center of the concentric circles.This is in line with the idea of Clean Architecture.On the other hand, looking at the dependencies, `Action` can also be placed in the center of the concentric circles.This can be viewed as a diagram with two centers of a circle (an ellipse).We will call this a "concentric ellipse diagram".In other words, Redux is a structure centered on `State` and `Action` .By rethinking the structure of Redux with a focus on dependencies, we may have been able to look at Redux from a different perspective.

## Redux and SOLID

Next, we will look at the relationship between Redux and SOLID.The assumption that SOLID is satisfied is affected by how Redux is implemented.Therefore, the nature of the Redux structure itself and how it is implemented need to be discussed separately.

From the discussion so far, we know that the major features of Redux are `State` and `Action` , which are independent of dependencies.How do we see this feature from a SOLID perspective?First of all, `State` does not depend on other elements, so adding new features will not affect other elements.This satisfies the principle of being "open to extensions" as specified in the Open-Closed Principle (OCP).So what about the principle of "must be closed to modifications"?Since `State` is dependent on other elements, modifications may affect other elements.This shows that we need to be careful when designing `State` so that the impact of modifications is small.The above story also applies to `Action` .

Next, let's consider `View` .`View` depends on `Action Creator` and `State` , but is not dependent on any element.Therefore, changes to `View` are unlikely to affect other elements.Expanding or modifying `View` will not affect the elements of other layers.Also, `View` is an element that is generally prone to change.By following Redux, you can protect `State` and `Action` from changes in `View` .On the other hand, there are some things to keep in mind.It is a dependency between `View` s.Dependencies between `View` s are created by `import` , but there are other implicit dependencies as well.It is a dependency via `State` .Even if `View` does not depend on each other directly, it may depend on each other via `State` .For example, if one view component is modified incorrectly and incorrect data is stored in `State` , other view components referring to the same data will be broken.In such cases, it is effective to use TypeScript's type checking to eliminate human errors as much as possible.

It is important to realize that `Action` and `State` are protected independently, making `View` even more susceptible.

# 3.7. Summary

In this chapter, we discussed the relationship between Clean Architecture and SPA.In recent years, SPA development has become more and more popular, and web frontend development has come to require advanced design.Clean Architecture is a technology that has been traditionally used in server-side and native applications.Web frontend will also be the focus of attention in the future.In adopting the Clean Architecture for SPA, the relationship with existing frameworks such as Redux is important.So we looked back at the basics of Redux and compared it to Clean Architecture.By analyzing the dependencies of Redux and fitting them into a concentric circle diagram, a new aspect of Redux was revealed.In the next chapter, we will finally discuss the implementation of a React application.

# Chapter 4. Scalable React

In this chapter, we will finally focus on React. By incorporating the concept of Clean Architecture into the design of React applications, we aim to create scalable applications.

## 4.1. React component

In this section, we will discuss React components alone.*1 In this section, we have classified the elements of React components into five categories as follows.*2

[*1] The reason I use the term "React component" instead of "component" here is that the term "component" is used differently in the context of Clean Architecture

[*2] Note that this classification is this book's own definition, not a general one

- State
- Props
- View Logic
- Renderer
- CSS

`State` and `Props` are common terms in React, so there's no need to explain them.`Renderer` means the `ReactElement` returned by the React component (or `render()`).`View Logic` represents the logic used to configure `Renderer` based on `State` and `Props` .The last one, CSS , probably needs no explanation.

An example of the above elements for Class component and Functional component is shown below.

List 4.1: Example in Class component

```
import React from 'react';

// Props
type Props = {
  size: number;
};

// State
type State = {
  number: number;
};

class ExClassComponent extends
React.Component<Props, State> {
  constructor(props: Props) {
    super(props);
    this.state = {
      number: 0
    };
  }

  render() {
    const { size } = this.props;
    const { number } = this.state;

    // View Logic
    const text = `size x number = ${size *
```

```
number}`;

    // Renderer
    return (
      <div
        // CSS
        style={{{
          fontWeight: 'bold
        }}
      >
        {text}
      </div>
    );
  }
}

export default ExClassComponent;
```

List 4.2: Example in Functional component

```
import React from 'react';

FC<{ const ExFunctionComponent: React.
  // Props
  size: number;
}> = ({ size }) => {
  // State
  const [number] = React.useState(5);

  // View Logic
  const text = `size x number = ${size *
number}`;

  // Renderer
  return (
    <div
```

```
      // CSS
      style={{{
        fontWeight: 'bold
      }}
    >
      {text}
    </div>
  );
};

export default ExFunctionComponent;
```

In this example, for the sake of brevity, CSS has been written directly in the `div` element.The part indicated by the comment corresponds to each element.The parts other than each element correspond to the glue code.Comparing the Class component and the Function component, we can see that the Function component has less glue code and can be written more concisely.

Now let's recall the SPA concentric diagram introduced in "3.3 Relationship between SPA and Concentric Diagrams" .Where would the React component be located on the SPA concentric circle diagram?The React component is the part of React that defines the view.Therefore, it is the outermost part of the circle in the SPA concentric circle diagram.In "3.3 Relationship between SPA and Concentric Diagrams" , we extracted the SPA equivalent from the product concentric diagram and applied the structure recursively to introduce the SPA concentric diagram.In addition, "3.6 Redux and Clean Architecture" created a "Redux concentric circle diagram" by organizing the dependencies of each element against Redux.Could we do the same for React components?React components have their own state and display logic, so let's organize them and map them as concentric circles.

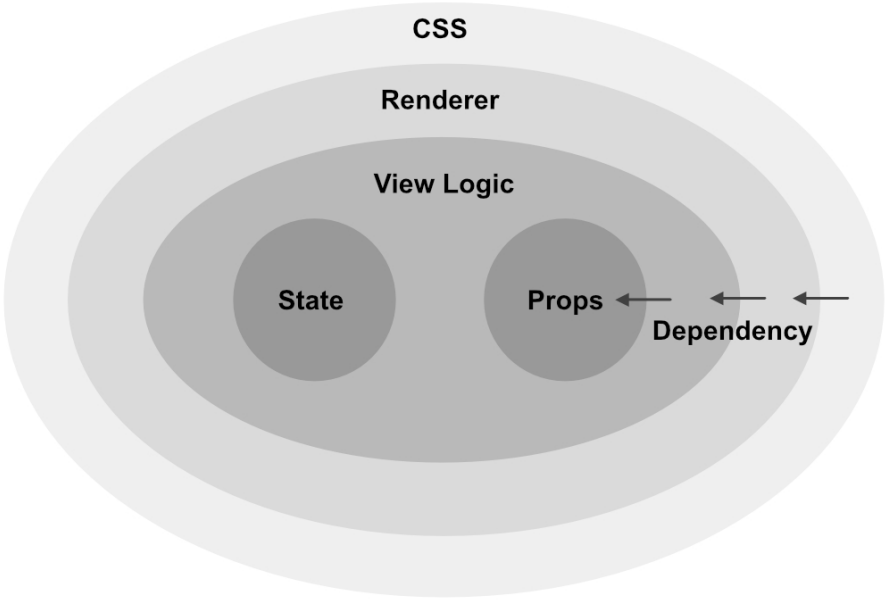


**Figure 4.1: Component concentric circles**

We'll call this concentric diagram "component concentric diagram *3 ".Dependencies between elements are important in drawing this diagrams, but what does dependency mean here?"Dependency Inversion Principle (DIP)" of "1.4 Design principles" says, "In TypeScript, a dependency is created by referencing it by `import` or `require` ".However, React components are customarily written in a single file and do not need to be `import` .So we'll extend dependency a bit and call it "a relationship that needs to be referenced by `import` if we force a file split (with proper granularity)".This way, dependencies can be defined even when they are completed in a single file.Now we can think about the dependencies between the React components.

[*3] This is technically a concentric ellipse, since the circle has two centers, but for simplicity's sake we'll call it a concentric circle diagram.See "Redux and concentric circles" of "3.6 Redux and Clean Architecture" for a description of concentric ellipse diagrams

The concentric circles diagram of Figure 4.1 can be explained in words as follows.

1. `State` is independent

2. `Props` is independent
3. `View Logic` depends on `State` and `Props`
4. `Renderer` depends on `View Logic`
5. CSS depends on `Renderer`

It is important to note that the above dependencies are not naturally derived from the direction of processing, but rather a design idea that can be used to create a React component that is resistant to changes.

Now, let's discuss the dependencies in 1 through 5 in detail.

## 1. State is independent

`State` is the "state" of the React component and should not be affected by other components.Therefore, it should not depend on other elements.If we follow the general implementation of React, `State` will be independent of the other elements.

## 2. Props is independent

Similar to `State` , `Props` is the "state" that React components receive from the outside, and should not be influenced by other components.Therefore, it should not depend on other elements.Also, `Props` has a role as an interface because it is a necessary element to manipulate this component from the outside.

## 3. View Logic depends on `State` and `Props`

`View Logic` generates the data required for display from `State` and `Props` .`State` and `Props` are the states that React components have, and `View Logic` is the process of determining what to display based on these states.

## 4. Renderer depends on `View Logic`

`Renderer` builds the `ReactElement` needed for display.The required data is received from `View Logic` .It is also possible to refer to `State` and `Props` directly, but here we will not refer to them directly but via `View Logic` .A concrete way to implement `View Logic` would be to create a hook function that receives props and state, and design `Renderer` to depend only on the return type of the hook function.Since this design has both advantages and disadvantages, it will be necessary to consider them before deciding whether to adopt it in actual development.In particular, it can be said that the disadvantages of using `View Logic` are accentuated for components with thin logic.

## Advantages

By limiting the dependency of `Renderer` to `View Logic` , the dependency becomes clear.Only `View` will be directly affected by changes to `State` and `Props` .Of course, `Renderer` is transitively dependent on `State` and `Props` , but by introducing `View Logic` , it becomes more abstract and resistant to change.

## Disadvantages

Even if `Renderer` uses the values of `State` and `Props` as is, it will still go through `View Logic` . This makes the implementation redundant.

### 5. CSS depends on `Renderer`

In general, `Renderer` will depend on `CSS` .For example, if you write the class name of `CSS` directly into a component, the component must know the details of the class name of `CSS` .Also, when used by CSS Modules with `import` , the component will depend on `CSS` .As such, the general implementation cannot satisfy this dependency rule.So, by reversing the dependency, we can make `CSS` depend on `Renderer` .For specific instructions, see "4.2 Dependency inversion of CSS"

Adhering to these dependencies allows for control when writing React components and improves the visibility of the code.It also protects the state of the component from change by placing elements that are susceptible to change outside of the concentric circles.In particular, since `Props` is the connection to the outside world, protecting it will reduce the impact on other React components.

Next, I will show you how to reverse the dependency of `CSS` .

---

# 4.2. Dependency inversion of CSS

There are three general ways to apply styles to React components.

1. Directly write the CSS class name
2. Css modules
3. CSS in JS

Here, we adopt CSS in JS because we need to use the interface in reversing the dependency.There are several libraries for CSS in JS, but in this section we will use `styled-components` .Of course, you can do the same thing with other libraries.

We will review the design of reversing CSS dependencies in the following three steps.

1. Write the styles in the same file.
2. Separate styles into other files
3. Separate the layers to inject the style

"1. Write styles in the same file" is a common way to write styles in component files.This design is useful when components and styles are tightly coupled and you want to manage them simultaneously.For example, application-specific components are often suitable for this design, as the logic and style of the component are often tightly coupled and not problematic.In "2. Separate styles into other files", components and styles are written in separate files.In this case, which style is applied to the component is written directly in the component's file. Dependencies cannot be completely eliminated because you need to import the style from the component.In "3. Separate so to inject style", in addition to the separation in 2., separate the code that combines the component and the style into a separate file.In Clean Architecture, the code that resolves such dependencies is called the main component.Now, let's take a closer look at these three designs.

## 1. Put the styles in the same file

In this design, React components and styles are written in the same file. This is a common method, and for many applications, this

method is sufficient.The directory structure is simple, as shown below.

List 4.3: Directory structure of design 1

```
src/components
└─ SampleCss.tsx
```

Place the files corresponding to the React components directly under the `components` directory.Depending on the size of your application, you may need to adopt Atomic Design or other methods to further organize your directory.Next, let's look at the contents of `SampleCss.tsx` .

List 4.4: SampleCss.tsx

```
import React from 'react';
import styled from 'styled-components';

const Text = styled.div`
  text-decoration: underline;
`;

const SampleCss: React.FC = () => {
  const text = 'This is s sample text';

  return <Text>{text}</Text>;
};

export default SampleCss;
```

`SampleCss` is a React component with the text "This is a sample text" decorated with the style `Text` .`Text` defines a style for underlining text.

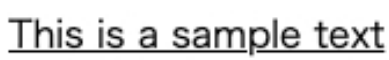


**Figure 4.2: Example of SampleCss's output**

## 2. Separate the styles into other files

The next step is to separate the style files to make it easier to switch between styles.This design can be used when you want to switch between multiple styles for a single component.Note, however, that the React component file still depends on the style file at this stage of design.The directory structure looks like this,

List 4.5: Directory structure of design 2

```
src/components
├─ SampleCss
│  ├─ index.tsx
│  ├─ styles1.ts
│  └─ styles2.ts
```

A directory is created for each React component, and `index.tsx` , `styles1.ts` , and `styles2.ts` are placed in same directory.`index.tsx` describes the implementation of the React component itself and its coupling with the style.Separate the style contents into `styles1.ts` and `styles2.ts` .In this example, two style files are provided to show how easy it is to switch between styles.Next, let's look at the contents of each file.

List 4.6: index.tsx

```
import React from 'react';
import { StyledComponent } from 'styled-
components';
```

```
// You can switch styles by changing this
import
import * as S from './styles1';
// import * as S from './styles2';

export interface Styles {
  Text: StyledComponent<'div', any>;
}

const { Text }: Styles = S;

const SampleCss: React.FC = () => {
  const text = 'This is a sample text';

  return <Text>{text}</Text>;
};

export default SampleCss;
```

List 4.6 shows a concrete example of `index.tsx` .In `index.tsx` , we are using a styled component called `Text` within a React component.It also has an internal interface for `Styles` .`styles1.ts` and `styles2.ts` define styles according to `Styles` .This way, `styles1.ts` and `style2.ts` are now dependent on the React component side.By doing the following, you can type-check whether the style file that you import satisfies the interface.

```
const { Text }: Styles = S;
```

List 4.7: styles1.tsx

```
import styled from 'styled-components';

export const Text = styled.div`
```

```
  text-decoration: underline;
`;
```

List 4.8: styles2.tsx

```
import styled from 'styled-components';

export const Text = styled.div`
  font-size: 2rem;
`;
```

`styles1.ts` is underlined in the same style as in design 1.`styles2.ts` is a style that doubles the size of text.To switch between these styles, change the file to import in `index.tsx` .This design makes it easy to switch between multiple styles.However, since the style file import is done in `index.tsx` , which implements the React component, the React component still depends on the style file.The React component still relies on the style file.The next step is to completely separate the React components from the style files.

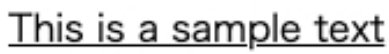


**Figure 4.3: `styles1.ts` table example**


**Figure 4.4: `styles2.ts` table example**

## 3. Separate the layers to inject the style

At this stage, the directory structure will look like the following,

List 4.9: design 3. directory structure

```
src/components
├─ SampleCss
│  ├─ index.ts
│  ├─ component.tsx
│  ├─ styles1.ts
│  └─ styles2.ts
```

`styles1.ts` and `styles2.ts` are exactly the same as in design 2.`components.tsx` is almost the same as `index.tsx` in design 2, but with a few modifications.

List 4.10: components.tsx

```
import React from 'react';
import { StyledComponent } from 'styled-
components';

export interface Styles {
  Text: StyledComponent<'div', any>;
}

export const injectStyle = (styles: Styles) =>
{
  const { Text } = styles;

  const component: React.FC = () => {
    const text = 'This is a sample text';

    return <Text>{text}</Text>;
  };

  return component;
};
```

The React component definition is wrapped in a function called `injectStyle` .This `injectStyle` is a function that takes a style definition and returns a React component.The type of style definition you receive at this time will be the one defined as `Styles` .`injectStyle` is called from `index.ts` shown below.

List 4.11: index.ts

```
// You can switch styles by changing this
import
import * as S from './styles1';
// import * as S from './styles2';
import { injectStyle } from './component';

const SampleCss = injectStyle(S);

export default SampleCss;
```

`index.ts` binds the defined React component to the style.Import `styles1.ts` and `styles2.ts` , and inject them by `injectStyle` .In this way, the implementation of the React component is now completely separated from the style details.On the other hand, the styles we need are defined as interfaces in `component.tsx` , so React components can get the styles they need.In other words, if a style definition is wrong, it can be detected by TypeScript's type checking.

In this section, we introduced a three-step design for reversing CSS dependencies.Which design is right for you will depend on the size and nature of your application, the policies of your team, and other factors, so you will need to make a choice while considering the tradeoffs.This is just an example, but by doing this, we were able to protect the React component from style changes by separating CSS and defining it according to the interface.

# 4.3. React component Relationships

In "4.1 React component" , we focused on the React component alone and discussed its design.In this section, we will review the design of a typical React application from the perspective of Clean Architecture in terms of the relationships of React components.In conclusion, we can say that although React component structure does not follow the principles of Clean Architecture, the high independence of its components and the separation of logic make it a practical design without any problems.

As you know, in React, you create components and combine them to create new components.It is recommended that each component be written as a function, and is required to have no internal state if possible.In addition, the input and output of the component is specified by `PropTypes` or TypeScript types, so that the calling side can easily understand the component specifications.By combining these highly independent components together, complex applications can be built.

## Classification of React components and their properties

Next, let's consider the relationship between the components.For the sake of clarity, let's divide React components into two types.There is no clear classification, but there are differences in degree.

1. Upstream components: Components that contain a lot of application logic and control the configuration and behavior of other components.
2. Downstream components: End components that are independent of other components

Upstream components contain a lot of application-specific logic and also depend on many other components.Because of this feature, upstream components are the core elements of an application, leaving little room for reuse.The downstream component contains very little application logic and is dedicated to the display of the screen. Also, the further downstream it is, the less dependence it has on other components.Because of this feature of downstream components, they are more likely to be components of the screen structure and are more likely to be reusable.These properties are a major feature of React and improve scalability in view development.

## Dependency and stability

Next, let's try to interpret the properties of these components in terms of dependencies and stability.If you look at the dependencies between React components, you can see that the parent component depends on the child components.Again, the upstream component will depend on many components, including indirect dependencies.On the other hand, downstream components have the property of being less dependent and more independent.

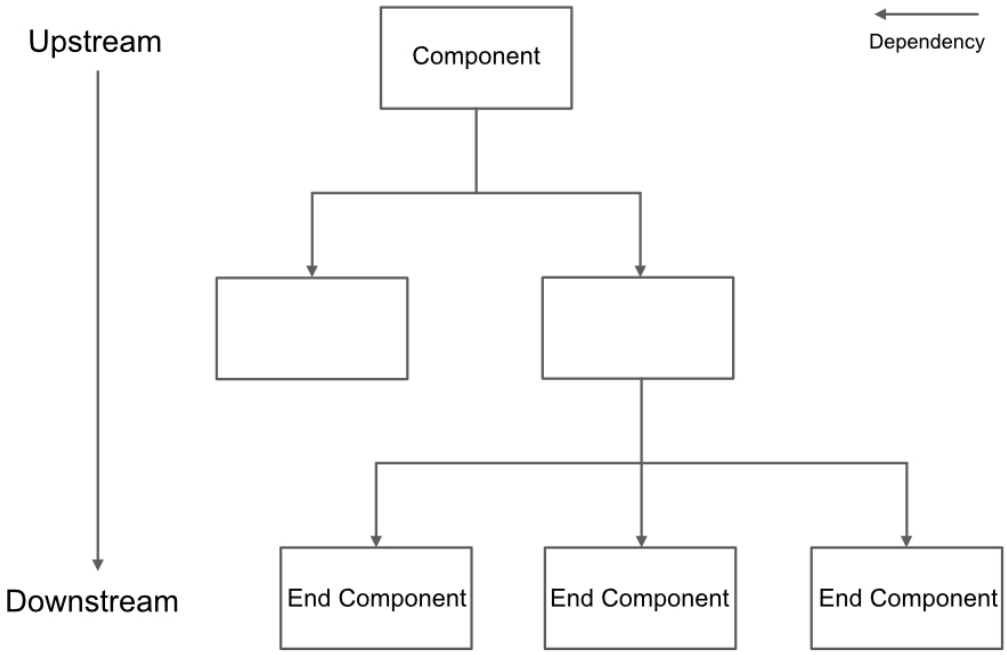


**Figure 4.5: Dependencies between components**

So what about in terms of stability?Within the overall application, the React components, which correspond to the views, are considered unstable because they change a lot, but let's consider the differences in stability within the React components.It can be said that the most frequent changes are in the areas related to design. In other words, the further downstream it is, the more unstable it needs to be.On the other hand, upstream components that contain a lot of application logic need to be stable in terms of the Clean Architecture principle.This is because if the upstream components, including the application logic, are unstable, the logic part will be affected by the other components and will change frequently.From the above story, you can see that general component design of React has the exact opposite relationship to Clean Architecture's concentric circles diagram.From the perspective of Clean Architecture, upstream components need to have fewer dependencies and be more stable, while downstream components can be unstable.However, in React, upstream components are more

unstable with more dependencies, and downstream components are more stable with fewer dependencies.

**Table 4.1: nature of the component**

| Classification | Reusability | Dependency | Actual stability | Ideal stability |
|---|---|---|---|---|
| Upstream | Low | Many | Low | High |
| Downstream | High | Less | High | Low |

## Is it effective to reverse the dependency?

Now, let's consider whether changing the way components are organized according to the Clean Architecture will result in a better design.In order to reduce the dependency of upstream components, we need to reverse the dependency between parent and child components.In other words, the interface is defined in the parent component, and the child component that implements it is injected into the parent component.This method has its advantages, of course, but it also has significant disadvantages.It means that the reusability of child components will be greatly reduced.Since the child component implements the parent's interface, it becomes difficult to use it under a different component.In addition, it is not practical to implement via an interface every time you combine many components.This is because the view itself is a layer that changes a lot when you look at the application as a whole.The Clean Architecture principle of reversing dependencies to protect business logic is most effective around business logic, which is the most stable part of the entire application. On the other hand, the disadvantage is greater around views, which are less stable.Frankly speaking, it would not be effective to reverse the dependencies of React component structure according to the principles of Clean Architecture.

## Why React components has high scalability?

So, why does React provide high scalability even when the upstream components have many dependencies?This is because each component has a high degree of independence and it is easy to separate the application logic from the design definition.In other words, even if there are many dependencies, they are designed to be low.As already mentioned, React tries to represent components with as pure a function as possible, and its interface (the `Props` given to the component) can also be properly specified.The user of the component does not need to worry about the internal state and behavior of the component as long as the interface is followed.In other words, even if there is an `import` relationship between components, the actual dependency is tenuous.In fact, a slight change in the display content or design of a downstream component will have little impact on the parent component or even more upstream components.Due to these properties, React component structure is considered to be highly scalable.

## Points to keep in mind for scalability

This concludes our discussion of why React's method of composing components can scale practically without following the principles of Clean Architecture.Finally, based on the above discussion, here are some points to keep in mind when designing components for React.Earlier, in describing the tenuous dependencies between React components, I wrote, "A small change in the display or costume of a terminal component will have little impact on its parent or further upstream components.This includes the assumption that if you have the right component design, it will scale.In other words, if the implementation is such that changes in the child components are propagated to the upstream components, the entire view will become unstable and will not scale.For example, if the design has many variables flowing downstream from the upstream in a so-called bucket relay, when a new variable is needed

in a child component, it will affect the upstream.You will need to design the relationships between components carefully to avoid having such strong dependencies.An example of how to weaken the dependencies between components is the Redux and Hooks APIs, which will be introduced later.

## Redux

In the previous section, I explained why it is better to make the dependencies between components as weak as possible.In this section, we explain how Redux can be used to weaken dependencies between components.Redux now supports the hooks API in v7.1.0, making it easier to write connections between React components and `State` .Before the introduction of the hooks API in Redux, there was also a way to design the separation of state connected and non-state connected components, as Dan Abramov, the author of Redux, mentions in his web article *4 Under this design policy, we needed to have as few components as possible connected to `State` and pass the state to downstream components in a bucket relay.As mentioned in the previous section, bucket relays create strong dependencies between components, which not only reduces reusability, but also causes instability in upstream components.

[*4] https://medium.com/@dan_abramov/smart-and-dumb-components-7ca2f9a7c7d0

On the other hand, as Dan Abramov adds in the same article, with the introduction of the hooks API, this separation is no longer necessary.The hooks API makes it much easier to connect components to `State` , so there is no need to separate them.Thus, the barrier of connecting components to `State` has been reduced, and it is now possible to connect `State` not only to upstream components but also to downstream components.This makes it possible to weaken the dependency between components by transmitting the state that needs to be passed from upstream to downstream via Redux's `State` .

As described above, the introduction of Redux weakens the dependencies between components, making it easier to compose components as clean from the perspective of Clean Architecture.

---

# 4.4. Hooks API

In this section, we will review the benefits and cautions of React hooks API from the perspective of Clean Architecture.The hooks API was introduced in React v16.8.In a nutshell, the hooks API allows functional components to manage state and lifecycle, which was previously done by class components.This feature eliminates the need to use the class component in most cases.The hooks API has made it possible to separate the logic part from the components, which has increased the reusability of the logic.This is possible because the hook function is component-independent.Of course, the separation and reuse of logic could have been achieved with patterns such as High Order Component (HOC), but the hooks API makes it easier and more flexible.

If logic cannot be separated and reused (or is difficult to reuse), it may be difficult to use the same logic for multiple components that require common logic.This is often solved by pushing the logic into the parent component and bucket-relaying the calculation results to the child components.With this method, the dependency between components becomes stronger.On the other hand, by using the hooks API to extract and reuse common logic in the hook function, parent components with common logic are no longer needed, and dependencies between components can be weakened.In this way, the hooks API makes it easier to configure components that are clean from the perspective of Clean Architecture.

Finally, I would like to mention some points to keep in mind when using the hooks API.As mentioned earlier, the hook function can be

reused only because it is independent of the component.In other words, if a hook function depends on a particular component, it will not be reusable.Or, by forcing reuse, you will create dependencies between components that are not really needed.A simple example of this would be if you have created a hook function that takes a type defined in a particular component A as an argument.In such a case, if we try to use this hook function in another component B, B will depend on the type of A.If you define a hook function, you will need to be careful not to depend on a specific component.

---

# 4.5. Summary

In this chapter, we discussed how to design React applications that scale from the perspective of Clean Architecture.First, we organized the components of a single React component based on their dependencies.In the React component, `State` and `Props` were the key elements and positioned at the center of the concentric circles diagram.Next, I showed you how to reverse the dependency of CSS , the most unstable element among the components of React components.In a typical application, it is not a problem if the component and CSS are tightly coupled, but we have presented a way to separate them by step so that they can be separated when necessary.Having discussed React components on their own, we then discussed the relationship between the components.React components are upstream to downstream dependent, which violates the principles of Clean Architecture.However, by making the components highly independent, we can weaken the dependency from upstream to downstream, and explain how practical scalability can be achieved without reversing the dependency.We also discussed Redux and the hooks API as a way to keep dependencies between components weak.When designing a React application, it is very important to keep it in line with the culture and context of React

development.If you design everything according to the Clean Architecture, you will be susceptible to version upgrades of React itself, and it will also become a barrier for other developers to catch up.We believe that by following the traditional culture and appropriately incorporating Clean Architecture design methods into it, we can get one step ahead in React application design.

# Conclusion

Thank you for reading this book to the end.As the demand for SPAs grows and React applications become larger and more complex, I believe it is becoming increasingly difficult to write code that scales by simply relying on React itself or state management frameworks like Redux.Recently, the Redux Toolkit, which is a collection of Redux best practices, has become available, and it can be said that it provides a guideline for Redux design.By following the design philosophy of the Redux Toolkit, we can follow the best practices presented by the official design around state management.On the other hand, the design of domain-specific application logic is a case-by-case basis and cannot be squeezed into a generic tool.And methodologies that apply different domain logic to the design of each product are increasingly required for SPAs as well.I hope that the contents discussed in this book will contribute as one of the solutions to this current situation surrounding SPA.

Originally, Clean Architecture was based on the knowledge gained from the development of large-scale products written in compiled languages such as Java.Therefore, when considering dependencies, it is important to consider whether compilation and deployment can be done separately.On the other hand, web applications such as React are basically built and deployed as a whole, and during development, there are ways to reflect only the change differences at high speed using hot reload.Therefore, this point is not as much of a problem as in compiled languages.Considering the background of the Clean Architecture, not all of it can be directly applied to React applications.There are some things that fall outside of what Clean Architecture assumes.

However, as Uncle Bob mentioned, "All architecture rules are the same! as mentioned in the text.Believing in these words, I believe that Clean Architecture can be applied to React application development, and even to web application development, even though it may take a slightly different form.This is the reason why I started writing this book.

Since the theme of this book is to reconsider React and Redux from the perspective of Clean Architecture, I have deliberately discussed some designs that are redundant to use in actual development.If you try to copy the contents of this book as is, you may end up with a code base that is more verbose than necessary, scalable, but not very flexible.However, having learned the principles of Clean Architecture and its applications through this book, I hope that you will be able to select and use the parts that are necessary for your own development environment.If so, then the goal of this book has been achieved.

---

# About Author

Born in 1991. Master of Social Informatics, Kyoto University, Kyoto (2016).After experiencing server-side development in the ad-tech domain, engaged in frontend development at a startup company.Currently working on a wide range of projects from product UI/UX design to architecture design and maintenance, mainly using React.

*Your gateway to knowledge and culture. Accessible for everyone.*